

Odoo QA Gate

An automated verification stage between "the developer says it is done" and "a human looks at it". It runs against each client's own cloud staging instance, on their real data and their full set of installed modules, and leaves that data as it found it. One database, no copies, and it hands the instance back whenever a person needs it.

Proposed repo odoo-qa-gate **Targets** Odoo 17, 18, 19 and forward **Runs on** the one staging database the host provides
Hosting Odoo.sh, Cloudpepper, or any managed platform **Merges into** hst-pmo-dashboard

01 What this actually is

A **gate**, not a test runner. It takes a task and a commit, and returns a verdict with evidence. A test runner answers "did my tests pass". The question you actually have is "is this safe to show the client".

WHAT CHANGED IN REVISION 3

Revision 2 still kept one escape hatch: where a diff was risky enough, it duplicated the client's staging database and ran a true twin comparison on the copy. On Odoo.sh, Cloudpepper, and the other managed platforms our clients actually run on, that is not available. There is **one staging database, provided by the host, with no way to clone it, duplicate it, or stand up a second one**. Revision 3 removes the escape hatch and everything that depended on it.

Two consequences follow, and they are the new material. The temporal baseline in section 8 is now the *only* baseline, with no true twin behind it, and three use cases become advisory rather than automated. And because there is exactly one database and people other than the gate need it, a run has to be able to stop, release the instance, and resume from where it left off. That is section 7, and it is the largest addition in this revision.

INPUT	WHERE IT COMES FROM	WHAT IT IS USED FOR
The task	Odoo <code>project.task</code> , already the source of truth in odoo-dev-loop	Acceptance criteria to demonstrate
The diff	<code>git diff base...head</code> in the client repo	Blast radius, which existing flows are now at risk
The staging instance	The client's own cloud staging Odoo, reached over authenticated JSON-RPC	Real data, real config, real installed module set
The suite	Version-controlled scenarios that accumulate over time	The regression floor

Out comes a verdict of **PASS** **REGRESSION** **UNMET** or **INCONCLUSIVE**, plus an evidence bundle: an ordered screenshot walkthrough with a value ledger, a regression diff, a residue report accounting for anything the run left behind, and a one-paragraph summary written for a project manager rather than a developer.

Why testing in place is genuinely better

Your instinct here is right, and it is worth being explicit about what it buys, because it is more than convenience:

- **The real installed module set.** A stack we assemble ourselves only ever contains the addons we knew to mount. A client's staging instance carries the third-party apps someone installed through the hosting panel, the theme module, the QuickBooks connector, the MUK web modules. Interaction bugs between your change and a module nobody remembered are a large share of what reaches clients, and they are invisible in a stack we assemble ourselves.
- **The real configuration.** Their chart of accounts, their taxes, their fiscal positions, their warehouse routes, their pricelists, their Studio fields. A change that is correct against a clean install can be wrong against their tax configuration specifically.
- **No golden-database pipeline to build or keep fresh.** That was two weeks of revision 1's roadmap and a permanent maintenance cost. It disappears.
- **The instance the client will actually accept on.** Evidence screenshots come from the same URL the project manager and the client would open themselves.

What it costs, stated up front

Four things get harder, and the rest of this document is largely about handling them:

1. **Writing without altering existing data** becomes the central engineering problem rather than a free consequence of throwing the database away. Section 5 and section 6.
2. **There is no twin, and now no way to make one.** You cannot hold the same database at two code versions at once, and the host will not give you a second copy, so the baseline is temporal and that is the end of the matter. It costs precision, and three use cases with it. Section 8.
3. **The instance is shared with people.** One database, one lock, and a developer who needs staging mid-run. A run must be suspendable and resumable or the team will route around it. Section 7.
4. **Safety is now an assertion about an instance we do not own** rather than a network we control. This is the part where a mistake is an incident, so it comes first. Section 3.

02 Seven load-bearing decisions

If any one of these is dropped the tool becomes a demo that gets abandoned. Four carry over from revision 1. Two came with in-place testing in revision 2. The seventh is new here, and it is what makes a single shared database workable rather than merely possible.

DECISION 1 · NEW

The regression suite runs inside a transaction that is rolled back

Scenarios execute in-process on the staging instance inside a savepoint, then roll back. Real data is read, written, asserted against, and released unchanged. This is the mechanism that makes "test on real data" and "do not affect existing data" both true at once, and it is the heart of the revision.

DECISION 2 · NEW

The instance opts in, or nothing runs

A staging instance must carry a hand-placed sentinel and pass a pre-flight safety audit before the gate will touch it. A production instance can never accidentally qualify. This is the single most important control in the design, because the failure it prevents is writing test data into a client's live system.

DECISION 3 · NEW

A run can be suspended and resumed

There is one database and it is shared with people. A run pauses at a checkpoint, releases the instance, and later resumes from where it stopped after re-auditing and re-fingerprinting the instance. Without this, one developer needing staging costs a whole run, and the team routes around the gate within a fortnight.

DECISION 4

Compare against a baseline, always

A result on its own cannot distinguish a new breakage from a pre-existing one, and unexplained failures get dismissed as noise. The baseline is now a nightly run rather than a parallel lane, with a data fingerprint attached so a stale comparison declares itself.

DECISION 5

Three execution tiers, matched to what may commit

Read-only probes, rolled-back transactions, and committed browser flows have completely different residue profiles and completely different coverage. Every scenario declares which tier it needs, and the tier decides what cleanup it owes. There is no fourth tier, because there is no second database to give it.

DECISION 6

Scenarios accumulate, they are not regenerated

AI proposes scenarios from the diff, a human ratifies them, they are committed to the client repo and run forever after. Regenerating tests every run gives flaky, unreviewable output and zero compounding value. This is what turns 40 untested client repos into tested ones.

DECISION 7

Test as several users, never only as admin

Access-rights regressions are among the most common in Odoo work and are completely invisible to an admin session. Every flow carries a persona list. Nearly free to implement, and it catches a whole failure class the manual process misses today.

03 The instance contract

In revision 1, safety came from infrastructure we owned: an isolated network with no egress, a database we had just neutralized ourselves. None of that is available now, and on a managed host there is no second database to retreat to either. The gate is about to write to a live instance belonging to a client, so what replaces it has to be louder, stricter, and refuse by default.

Opt-in by sentinel

An instance becomes eligible only when a human deliberately marks it. The marker is a config parameter set by hand, once, on that instance:

```
ir.config_parameter
  key   = hst_qa.instance_role
  value = staging:<client_id>:<iso8601 timestamp>:<who set it>
```

The probe reads this before doing anything else. No sentinel, no run. The point is the direction of the default: an instance has to be walked up to and opted in, and there is no configuration mistake on our side that makes a production instance eligible. A wrong URL in a client config file then produces a refusal, not an incident.

THE PRE-FLIGHT AUDIT · EVERY ONE OF THESE REFUSES THE RUN

1. **Sentinel missing, malformed, or naming a different client** than the run's client id.
2. **Database name not on the client's allowlist pattern.** Configured per client, for example `*_staging` or `staging_*`. Belt to the sentinel's braces.
3. **Any active `ir.cron` record.** A staging instance should not be running scheduled jobs, and crons firing mid-run both corrupt results and can act on rolled-back records.
4. **Any active `ir.mail_server` or `fetchmail_server`.** This is the one that emails four thousand customers if you get it wrong.
5. **Any `payment.provider` not in test mode.**
6. **A `web.base.url` that resolves to a production hostname,** checked against the client's known production domain.
7. **Integration config parameters holding what look like live credentials,** matched by key pattern against a per-client list of the endpoints that module set touches.
8. **A module declaring an outbound integration with no stub registered** for it. See the convention below.

The audit result is recorded in the bundle with every check and its outcome, so a refusal is self-explanatory and a pass is auditable after the fact. Re-audited at the start of every run, never cached, because a client's hosting provider can re-enable crons or mail during a platform update without telling anyone.

Guards inside the run, as defence in depth

The audit proves the instance's resting state. It cannot prove what your code does once it executes, so the probe additionally patches, for the duration of each QA transaction only:

PATCHED	BEHAVIOUR DURING A QA TRANSACTION	WHY
<code>IrMailServer.connect</code> and <code>send_email</code>	Raises, and the attempt is recorded as a finding	Catches code that sends mail inline rather than queueing it for cron
<code>requests.Session.request</code>	Raises unless the target matches a registered stub	Catches outbound integration calls. An unstubbed call is a finding, not a silent success
<code>Cursor.commit</code>	Raises	An explicit commit would escape the rollback. It is also a code smell worth reporting on its own
<code>Cursor.execute</code> on DDL	Raises	Schema changes do not belong inside a scenario, and some do not roll back cleanly

THE CONVENTION WORTH ESTABLISHING BEFORE ANY OF THIS IS BUILT

Every custom module that talks to the outside world declares it in its manifest and ships a stub descriptor. The gate refuses to run a scenario touching a module that declares an integration without one. That converts a recurring human check into a structural one, and it is the highest-value convention to adopt early. The QuickBooks connector, the ZK attendance sync, and the ESC/POS printer bridges are exactly the modules that need it.

Per-client capability flags

Access to client hosting is not uniform, and on managed platforms it is deliberately restricted: Odoo.sh and Cloudpepper hand you a web console and an RPC endpoint, not a shell and not a database. So the design's floor is **authenticated JSON-RPC and nothing else**, which is precisely why the probe module exists rather than a collection of shell scripts. Everything the gate needs to be correct sits on that floor. The one capability above it is a convenience, and no use case depends on it:

CAPABILITY	CHANNEL	UNLOCKS
rpc	HTTPS JSON-RPC with an Odoo API key	Everything in tiers 1 to 3. The baseline. Assumed present for every client
browser	HTTPS to the same host from Playwright	The evidence plane. Present wherever RPC is
shell	SSH, PEM key or Tailscale. Absent on most managed platforms	Server log retrieval for tracebacks, module upgrade without the hosting panel, asset rebuild. Convenience, never capability

There is no **db** capability, and this is the defining constraint of revision 3. Odoo.sh, Cloudpepper, and the managed hosts our clients run on do not hand out a Postgres superuser connection, and none of them will let you **CREATE DATABASE** from a copy of the staging one. Revision 2 assumed that access existed for a minority of clients and built an escalation path on it. That path is removed rather than made optional, because a feature available to three clients out of forty is a feature nobody builds scenarios against.

The upshot is a simpler and more honest promise: **every client gets exactly the same gate**. There is no tier of client that gets better answers, and nothing in the roadmap is gated behind hosting access somebody has to negotiate. What is lost, stated plainly rather than buried in section 18, is module uninstall testing, migration script verification, and version upgrade dry runs. Section 18 covers what replaces them.

04 Architecture

Two processes we run, plus a small module installed on each client staging instance. There is no container orchestration, no database provisioning, and no infrastructure of ours between the runner and the client's instance. Everything the gate does, it does over the network to a host somebody else operates.

COMPONENT	LANGUAGE	RUNS ON	RESPONSIBILITY
Control plane api/	TypeScript, Express	Railway, next to the PMO dashboard	Run queue with a per-client lock, run history, Odoo task read and chatter write, report hosting, auth
Runner runner/	Python 3.12	One host, ours	Claims jobs, parses diffs, deploys the branch to staging, calls the probe, drives Playwright, assembles the bundle
Probe module hst_qa_probe	Python, an Odoo module	Installed on each client staging instance	Runs scenarios in-process inside a rolled-back savepoint, enforces the guards, reports the pre-flight audit, records the creation registry

The runner does not need to be co-located with anything, because there is nothing local for it to reach: no container socket, no database to copy, no per-client infrastructure. It holds credentials and speaks HTTPS. One modest host serves every client, and adding a client is a config entry rather than a provisioning job.

THE CONTRACT

```

POST  /api/runs          # enqueue: task_id, client, base_sha, head_sha
GET   /api/runs/:id      # status, phase, live log tail

# runner polls, authenticated with X-Runner-Token
GET   /runner/jobs/claim  # long-poll, returns one job or 204
POST  /runner/jobs/:id/result # verdict, findings[], residue[], bundle

# probe, on the client staging instance, over authenticated JSON-RPC
hst.qa.probe  audit()          -> pre-flight report, no side effects
hst.qa.probe  fingerprint()    -> data + module + config fingerprint
hst.qa.probe  run_scenario(spec) -> results, rolled back, zero residue
hst.qa.probe  run_committed(spec) -> results + creation registry, for tier 3
hst.qa.probe  cleanup(registry) -> reverses a committed run, reports residue

```

One run at a time, per client

This is the throughput consequence and it needs stating plainly. A client has one staging instance, and a run needs that instance to be running one specific commit. So runs serialize per client. The control plane holds a per-client lock and queues behind it.

In practice this is fine: a run is 12 to 20 minutes, so a client absorbs around 20 runs a day, well above what five developers generate against a single client. But it does mean a run cannot start while a developer is manually clicking through the same staging instance, and the lock has to be visible in the dashboard so people can see why they are waiting.

Waiting is only acceptable if it is bounded, and on a single shared database it is not bounded by default: a run that has to finish before anyone else can work is a run people will kill. So the lock is **surrenderable**. A run pauses at a checkpoint, hands the instance over, and resumes later without losing what it had already proved. That mechanism is section 7, and it is the reason one instance per client is a constraint rather than a bottleneck.

A PREREQUISITE IN ODOO-DEV-LOOP WORTH CHANGING FIRST

odoo-dev-loop's client config defaults to `branch_mode: "shared"`, where every task lives on one working branch. If two tasks are sitting on that branch together, the gate deploys both at once and **a regression cannot be attributed to a task**. The verdict would be correct and useless.

`branch_mode: "per_task"` is effectively a prerequisite for clean attribution, so switch clients over as they come onto the gate. Where a client has to stay on `shared`, the gate can still run, but its findings must be reported against the branch rather than the task, and that is worth knowing before someone argues about whose change broke it.

05 The probe and rollback

This section is the mechanism the whole revision rests on. Everything else is plumbing around it.

Why it has to be in-process

Over JSON-RPC, every call is its own transaction and commits when it returns. There is no way to begin a transaction, make twelve calls, and roll back, because the first call has already committed. So a scenario cannot be a sequence of RPC calls if it is going to be undone.

Instead the scenario travels as data in a **single** RPC call, and executes inside the Odoo process where one transaction spans the whole thing. That transaction is wrapped in a savepoint which is unconditionally rolled back. Odoo's own test framework has worked this way forever, with `TransactionCase` doing exactly this against a real database. The probe is that idea, reachable remotely.


```
# hst_qa_probe/models/probe.py (shape, not final code)

class _Rollback(Exception):
    """Sentinel. Never escapes run_scenario."""

class QaProbe(models.AbstractModel):
    _name = "hst.qa.probe"

    @api.model
    def run_scenario(self, spec):
        self._assert_sentinel()          # refuse if not opted in
        results, log = [], []

        self.env.flush_all()            # nothing pending leaks into the savepoint
        try:
            with self.env.cr.savepoint():
                with self._guards(log): # mail, http, commit, DDL
                    results = self._execute(spec)
                raise _Rollback()
        except _Rollback:
            pass
        finally:
            self.env.invalidate_all()    # the cache still holds rolled-back values

        return {"results": results, "guard_events": log}
```

Two lines there are easy to leave out and both cause real bugs. `flush_all()` before the savepoint, or pending ORM writes from earlier in the request get swept into it. And `invalidate_all()` after the rollback, or the environment cache keeps serving values that no longer exist in the database, which produces assertion results that are confidently wrong.

What rollback does not undo

Rollback is close to complete, not complete. These are the leaks, and each needs a decision rather than a hope:

LEAK	WHY	HANDLING
Postgres sequences	<code>ir.sequence</code> with the standard implementation uses a real Postgres sequence, and <code>nextval</code> is deliberately non-transactional. Confirming an order consumes its number whether or not you roll back	Numbering gaps on a staging instance are harmless, so the default is to accept them and record which sequences were touched. Optionally the probe captures <code>last_value</code> before and restores it after, which is safe on staging because concurrency is low but is not safe in general
Explicit commits in the code under test	Any <code>cr.commit()</code> ends the transaction and makes everything before it permanent	The guard patches <code>commit</code> to raise. The scenario fails with a clear message, and <i>the commit itself is reported as a finding</i> , because inline commits in business logic are a defect worth fixing
Filestore writes	<code>ir.attachment</code> writes its file to disk before the transaction ends	The database row rolls back, leaving an orphan file that Odoo's own attachment garbage collection removes. No action needed beyond knowing why the filestore grows slightly
Anything outside Postgres	Emails, webhooks, external APIs, printed documents	Not a rollback problem at all. Handled by the pre-flight audit and the in-transaction guards in section 3, which is why those are not optional
Lock contention	A transaction that writes a hot row holds a lock until it rolls back, which can block a person using staging at that moment	One scenario per transaction, so transactions last seconds. <code>lock_timeout</code> and <code>statement_timeout</code> set on the probe's work so it fails fast rather than blocking a human, and a timeout reports as INCONCLUSIVE rather than as a failure

The honest summary: for ORM-level business logic, which is the overwhelming majority of what your custom modules do, rollback isolation is effectively complete. The leaks are enumerable, each is either harmless or reported, and none of them modify existing client records.

The creation registry, for what cannot roll back

Tier 3 flows drive the browser, so they commit, and they need a different mechanism. During a committed run the probe hooks `create` across all models and records every `(model, id)` it produces, along with the ordering.

That registry is a reliable ledger of everything the run made, which name prefixes and marker tags are not. Cleanup walks it in reverse creation order. Where `unlink` is refused, and Odoo refuses plenty, the probe applies the documented reversal for that model and, failing that, reports the record as residue with its id and the reason. Section 6 covers what that means in practice, and section 14 covers how it appears in the bundle.

06 Isolation tiers

Every scenario declares one tier. The tier decides what the scenario may do, what it can verify, and what cleanup it owes. Coverage rises down the list and so does residue, which is the whole trade.

TIER 1
READ-ONLY

Probes that never write

View resolution and arch validation across every inheriting view, report rendering against existing records, ACL and record-rule evaluation per persona, query-count measurement, search and read performance on real data volumes.

Residue none Cost seconds Misses anything requiring state change

TIER 2
ROLLBACK

Write, assert, roll back

The regression suite lives here, and it is where most of the value is. Full business flows over the ORM: create, confirm, deliver, invoice, pay, cancel, reverse. Constraint and compute behaviour on real data. The persona matrix. Everything in section 5.

Residue sequence numbers only Cost 0.5 to 3s per scenario

Misses everything visual, and anything spanning multiple HTTP requests

TIER 3
COMMITTED

Browser flows, cleaned up afterwards

The evidence plane. Playwright drives the real web client, so every click commits. This is the only tier that produces screenshots, and the only one that catches view rendering, OWL errors, and widget behaviour. 4 to 10 flows per run, not a regression suite.

Residue bounded, reversed, and reported Cost 30 to 120s per flow

Misses nothing visual, but too slow and flaky to run broadly

THERE IS NO TIER 4

Revision 2 had one: duplicate the staging database, run a true twin comparison or a destructive test on the copy, drop it. On a managed host there is no copy to be had, so the tier is gone rather than optional.

What it took with it: module uninstall testing, migration script verification, and version upgrade dry runs. Those are not merely slower now, they are **outside what the gate can do**, and sections 10 and 18 say so where a reader would otherwise assume coverage. Pretending a temporal comparison substitutes for a twin would be the more comfortable choice and the wrong one.

Tier 3 is where "do not affect existing data" needs care

Tiers 1 and 2 satisfy the constraint outright. Tier 3 cannot, because a browser click is a committed transaction, and there is no mechanism that undoes it invisibly. So the goal shifts from zero residue to **bounded, reversed, and accounted-for residue**, with three rules:

1. **Nothing existing is ever modified.** Tier 3 flows may only create new records and act on records they created. A flow that needs to edit an existing customer order creates its own order instead. The registry makes this enforceable rather than aspirational: any write to a record id not in the registry aborts the flow.
2. **Cleanup runs in reverse, and reports what it could not do.** Draft records unlink cleanly. Confirmed orders cancel then unlink. Posted journal entries cannot be deleted at all, only reversed, so they are reversed and both entries are reported as residue. Done stock moves are the same story. This is Odoo behaving correctly, and pretending otherwise would be the mistake.
3. **Residue is visible in the bundle**, per record, with model, id, and why it survived. Never silent. A project manager reading the report can see exactly what the run added to staging, and a scheduled sweep reconciles the registry against the instance weekly to catch anything cleanup missed.

THE OPTIONAL STRONGER ANSWER, PER CLIENT

For clients whose staging books must stay pristine, tier 3 can run inside a dedicated **QA company** in the same database: its own journals, warehouse, and sequences, with a QA user who can only see it. Odoo's own multi-company record rules then do the isolation, and posted entries never touch the client's books at all.

The trade is real and worth understanding before choosing it. A QA company is not the client's company, so you are no longer testing against their exact journal, tax, and warehouse configuration, which was one of the main reasons to test in place. It also needs per-client setup, and on a single-company client it means enabling multi-company, which some custom modules handle badly.

Recommendation: default to the client's real company with registry cleanup, and reach for a QA company only for accounting-heavy clients or where someone objects to reversal entries appearing on staging.

07 Pausing and resuming a run

There is exactly one staging instance per client and people other than the gate need it. Section 4's per-client lock is what makes runs correct, and it is also what makes them obstructive: without a way out, a developer who needs staging must either wait out a twenty-minute run or kill it and lose everything it had already proved.

So a run is not a single indivisible operation. It is a sequence of checkpointed units that can be suspended between any two of them, releasing the instance, and picked up later from the last checkpoint rather than from the

beginning. This section is new in revision 3, and it is the direct consequence of never having a second database to fall back on.

What pausing actually releases

Two different things get asked for when somebody says they need staging, and conflating them produces a feature that does not help:

MODE	WHAT IT DOES	COST TO RESUME	WHEN
release	Drops the per-client lock and stops executing. The task branch stays deployed. The instance is free to use, running the code under test	Seconds. Re-acquire the lock and carry on	The default. Usually the person asking for staging wants to look at <i>this</i> task's work anyway, which is already deployed
yield	Drops the lock and redeploys the branch the instance carried before the run started , restoring it to its resting state	Two to four minutes, because the task branch has to be deployed again	Somebody needs staging for unrelated work, or for a different task's branch

Both are real pauses: results already collected are kept, and the run resumes from its checkpoint. The difference is only what state the instance is left in, and the bundle records which mode was used so a reader knows what the instance looked like during the gap.

Where a run may pause, by tier

A pause request does not interrupt work in flight. It is honoured at the next checkpoint boundary, and what that boundary costs differs sharply by tier, which is the reason section 6's tiers had to be explained first.

TIER	CHECKPOINT BOUNDARY	WORST-CASE WAIT	WHAT IS AT RISK
Tier 1 read-only	Between probes	Under a second	Nothing. Probes write nothing, so a pause is free and a repeat is harmless
Tier 2 rollback	Between scenarios, on the rollback that already ends each one	Around 3 seconds	Nothing. The transaction boundary <i>is</i> the checkpoint boundary, which is the single luckiest thing about this design
Tier 3 committed	Between flows, never inside one	Up to 120 seconds	Real. A flow interrupted halfway has created records and not yet cleaned them up. See below

Tiers 1 and 2 get this almost for free. Every scenario is already a self-contained call that ends in a rollback, so "stop after this one" needs a flag check between calls and nothing else. The checkpoint is the list of scenario ids that have completed and their results.

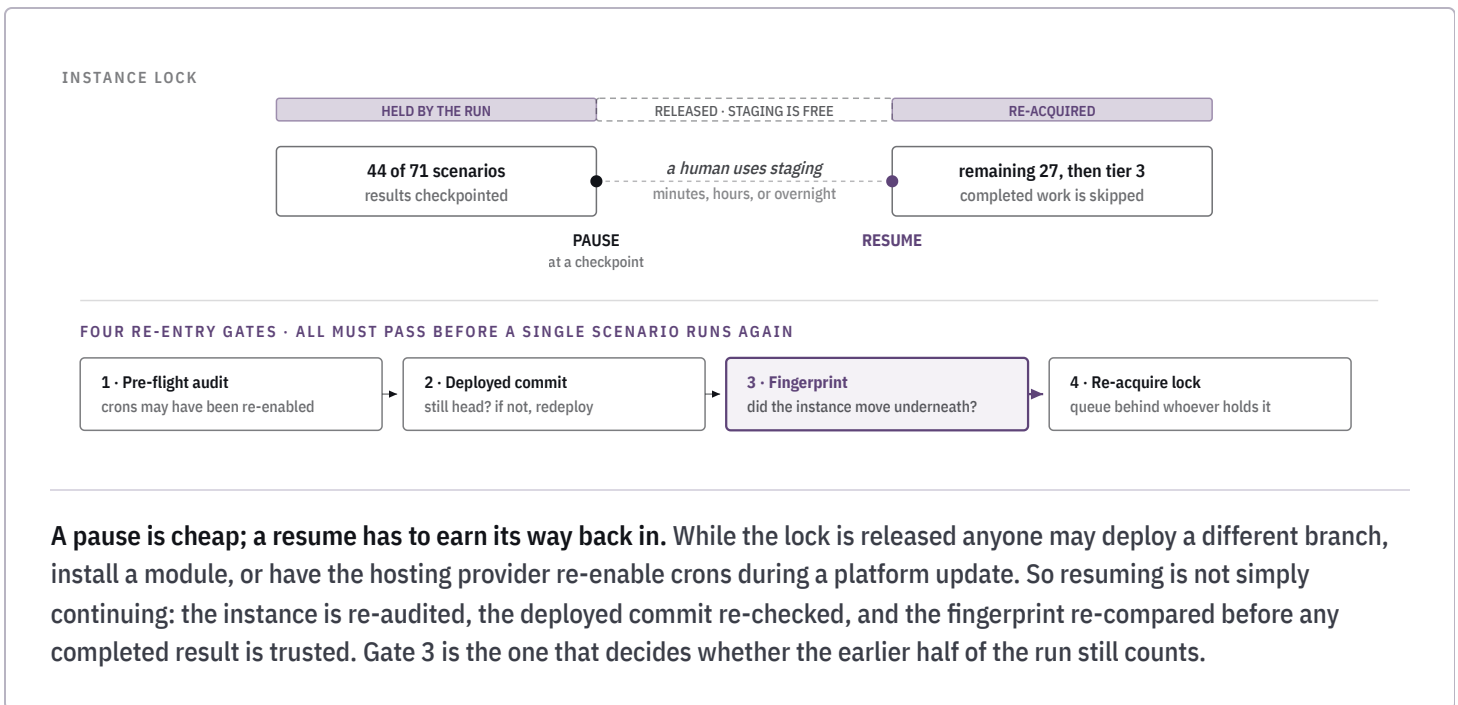
Tier 3, where pausing costs something

A browser flow commits as it goes, so an interrupted flow leaves records behind that its own cleanup step has not reached yet. Three rules, in the same spirit as section 6:

1. **A pause request never lands mid-flow.** The current flow runs to completion, including its cleanup, and the pause is honoured after. A flow is 30 to 120 seconds, so the wait is bounded and predictable, and the operator is

shown the estimate when they click.

2. **An abort is different from a pause, and is offered separately.** Where somebody needs the instance immediately, the flow is abandoned and the creation registry is walked in reverse right then. That reclaims the instance in seconds rather than minutes, at the cost of re-running the whole flow on resume rather than skipping it.
3. **A paused run reports its interim residue.** Records created by completed flows are already cleaned; what remains is whatever the tier's normal reversal rules could not remove. The pause writes a partial `residue.json` so the instance's state during the gap is documented rather than assumed, and a paused run that is never resumed is swept by the weekly reconciliation the same as any other.



The checkpoint

Written to the control plane after every completed unit, not held in the runner's memory, so a runner crash or a machine reboot is recoverable in exactly the same way an operator pause is:

```
{
  "run_id":      "run-8421",
  "phase":      "tier2",
  "completed":  ["sale.line_discount", "mrp.unbuild_finished_order", ...],
  "results":    [...],                # everything asserted so far
  "plan_sha":   sha256 of plan.json,  # the plan may not change under a resume
  "head_sha":   the commit that was deployed when work started,
  "pre_pause_ref": the branch to restore under `yield`,
  "fingerprint": the instance fingerprint at the moment of the pause,
  "registry":   tier 3 creation registry, with cleanup state per record,
  "paused_at":  timestamp,
  "expires_at": timestamp + the client's resume window
}
```

The `plan_sha` is worth more than it looks. A resumed run must execute the plan that was approved before the pause, not a freshly generated one. If the task description changed in the gap, that is a new run rather than a continuation of this one, and the checkpoint refusing to match is what makes that automatic rather than a matter of somebody noticing.

What resume checks, and what happens when a check fails

Gate 3 is the interesting one, because it reuses the fingerprint machinery from section 8 with no new mechanism at all. The comparison is between the fingerprint taken at the pause and the one taken at the resume, and the response is graded rather than binary:

WHAT MOVED DURING THE PAUSE	RESPONSE ON RESUME
Nothing. Fingerprints match	Resume from the checkpoint. The normal case, and the only one that is silent
Row counts moved within tolerance	Resume, noted in the bundle. Somebody used staging, which is exactly what the pause was for
A different branch was deployed and left there	Redeploy head, then resume. Results already collected still stand, because they were collected against head
Module set or config changed	Completed results are marked SUSPECT rather than discarded. Scenarios whose blast radius touches the changed module are re-run; the rest are kept. This is the graded response that makes a long pause survivable
The pre-flight audit now fails	Verdict INSTANCE UNSAFE , exactly as at the start of a run. No blame, lead alerted, nothing further written
The checkpoint is past <code>expires_at</code>	Results are advisory only and the run restarts. Default window 24 hours, per client
<code>plan_sha</code> no longer matches the task	Refuse to resume. The task changed, so this is a new run

WHY PARTIAL INVALIDATION RATHER THAN STARTING OVER

The tempting simplification is to throw the checkpoint away whenever anything moved. It is also the one that makes the feature useless: an overnight pause on a busy client almost always sees *something* move, so a strict rule means every long pause costs a full re-run and people stop pausing.

Re-running only the scenarios whose blast radius intersects what changed is the same selection logic section 10 already performs against a diff, pointed at a fingerprint delta instead. No new machinery, and it keeps the common case cheap.

Who can pause, and the request queue

Three ways a run pauses, and the third is the one that makes this a feature rather than a control:

- **An operator clicks pause** in the dashboard, choosing `release` or `yield`. The estimated wait to the next checkpoint is shown before they commit to it.

- **The runner pauses itself** on a lock timeout, a deploy failure, or an instance that stops answering. That is a pause rather than an `INCONCLUSIVE` verdict, because the work already done is worth keeping and the cause is often transient.
- **Somebody requests the instance.** A developer clicks "I need staging" on the client's page rather than hunting down whoever started the run. The request is queued, the run pauses at its next checkpoint, and the requester is notified when the instance is theirs. **This is what replaces the current practice of messaging the team and hoping.**

A paused run holds its place in the queue, so resuming does not mean starting again behind three other clients' work. Section 4's lock is unchanged: still one run at a time per client, still visible in the dashboard, now with a legitimate way to hand the instance over mid-run.

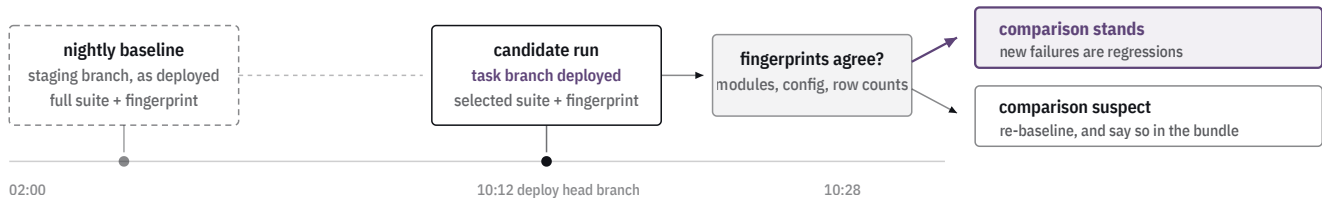
ADDITIONS TO THE CONTRACT

```
POST /api/runs/:id/pause      # {mode: "release" | "yield", abort_flow: bool}
POST /api/runs/:id/resume    # re-audits, re-fingerprints, re-acquires the lock
GET  /api/runs/:id/checkpoint # what would be skipped, and what would be re-run
POST /api/clients/:id/request # "I need staging" – queues a pause at the next checkpoint
```

Verdict `PAUSED` joins the table in section 15. It blocks nothing, moves no task, and carries the checkpoint summary and interim residue so a paused run is as readable as a finished one.

08 Baseline without a snapshot

Revision 1 ran base and head lanes side by side on identical database clones, which made "this scenario passed before your change and fails after it" airtight. Revision 2 kept that as an escalation for high-risk diffs. Revision 3 has neither, because the host will not give us a second database, so the airtightness has to be replaced with something weaker and honest about being weaker.



The baseline is temporal, it declares its own reliability, and there is nothing behind it. The nightly run records a fingerprint of the instance alongside its results. If the fingerprint has moved by the time the candidate runs, meaning a module changed, a config parameter changed, or key table row counts moved beyond tolerance, the comparison is flagged suspect rather than quietly trusted. In revision 2 a high-risk diff could escape to a database clone at that point. It cannot now, so the honest response is a fresh baseline and a bundle that states the confidence it actually has.

The fingerprint

Cheap to compute over RPC, and it is what turns a silent trust assumption into an explicit check:

```
{
  "modules": sha256 of (name, installed_version, state) for every installed module,
  "config": sha256 of ir_config_parameter keys and values, excluding our own,
  "views": count of ir_ui_view + max write_date,
  "volumes": row counts for the models in the blast radius, with a tolerance band,
  "sentinel": the opt-in value, so a re-opt-in is visible,
  "taken_at": timestamp
}
```

The `modules` hash is the important one. It catches somebody installing an app through the hosting panel between the baseline and the candidate, which is otherwise an invisible cause of a mysterious new failure.

Scenarios must be drift-immune to be comparable

Because the data underneath moves, scenario authoring gains one hard rule and one classification:

- **Build your own fixtures.** A tier 2 scenario creates the records it needs inside its transaction rather than asserting on the state of existing ones. Resolving "any customer with a sales history" by query is fine. Asserting that customer S00312 has a total of 4,120.50 is not, because someone will edit it on staging next Tuesday.
- **Data-dependent scenarios are tagged and treated differently.** Some checks are genuinely about existing data: how many records have a NULL in the new stored field, does the report render for the client's five weirdest historical orders. Those are valuable and they are inherently drift-sensitive, so they are compared only within a single run window, never against last night's baseline.

This rule was good practice in revision 1 and is a requirement now. It is worth enforcing in the scenario schema rather than in review comments.

When the baseline is stale, and what to do

SITUATION	RESPONSE
Fingerprint matches	Comparison stands. Normal case, most of the time
Volumes moved within tolerance, everything else matches	Comparison stands, noted in the bundle
Module set or config changed	Comparison flagged suspect. New failures report as NEEDS RE-BASELINE rather than REGRESSION , and the runner queues a fresh baseline immediately
No baseline exists for this instance yet	First run is baseline-only and blocks nothing. Advisory by construction
Diff carries a HIGH risk signal	Take a fresh baseline immediately before deploying head, rather than relying on last night's. It costs the suite's runtime twice but removes most of the drift window, and it is the closest thing to a twin that a single database allows
The run was paused and later resumed	The pause-time and resume-time fingerprints are compared as well, and completed scenarios touching anything that moved are re-run. Section 7

THE TIGHTEST BASELINE A SINGLE DATABASE PERMITS

Worth naming, because it is the mitigation that replaces tier 4. For a high-risk diff, run the selected suite against the *current* staging code first, deploy head, then run it again. Both halves see the same database minutes apart rather than hours, and no third party has touched it in between because the gate holds the lock across both.

That is not a twin: the two halves still run at different moments, and a cron or a colleague could in principle intervene. But it collapses the drift window from overnight to minutes, it needs no hosting access at all, and it costs one extra suite runtime on the small fraction of diffs that warrant it.

09 The client knowledge base

The impact graph in the next section is only as good as its answer to one question: what else is installed on this client's instance, and what does it touch? Getting that answer wrong is how a change ships having been tested only against the module it lives in.

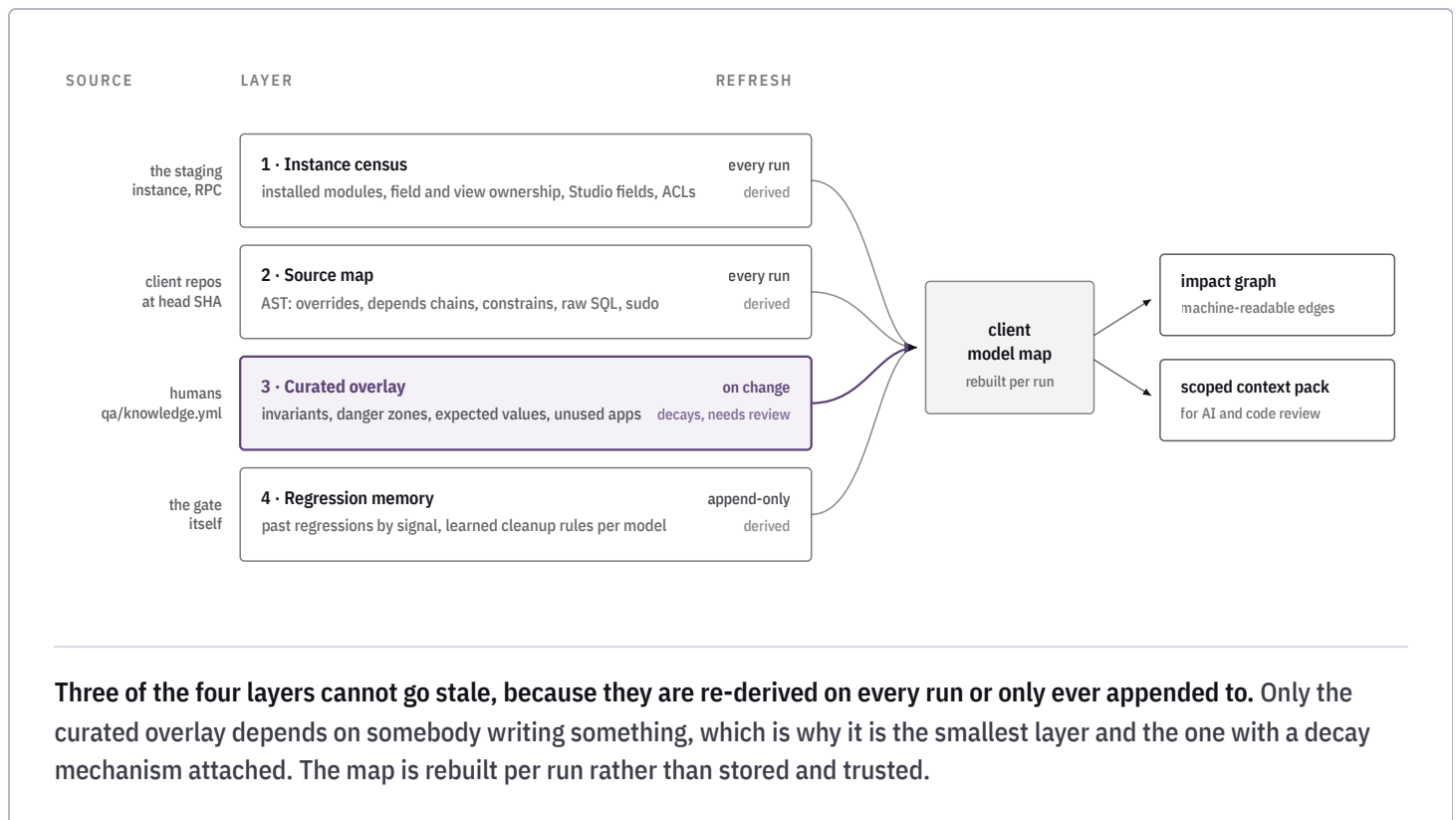
The mistake to avoid is treating this as a document somebody maintains. It is four layers with four different lifecycles, and only one of them involves a human writing anything.

What already exists, and its two gaps

odoo-dev-loop already carries a per-client knowledge base at `state/knowledge/<client_id>.json`. It holds manifest metadata per module, each module's `CLAUDE.md` verbatim, regex-extracted `_name` and `_inherit` declarations plus field definitions, and free-form user notes. `code_review.py` consumes it through `render_for_prompt`, which is already well built: it renders detail blocks for the modules intersecting the diff and a brief index for everything else, capped at 30 KB.

Its header says it is deliberately not a vector database, not LLM summaries, and not a file watcher. That restraint was the right call and it should survive this. Two things do need fixing:

- **It knows the repository, not the instance.** It has no idea which third-party modules are installed, which Studio fields exist, or which module patches which view. That is exactly the context the gate needs, and it is precisely what testing in place makes available.
- **Extraction is regex, and staleness is flagged rather than repaired.** `is_stale` compares the repo HEAD against `repo_sha_at_build`, and the poller deliberately does not auto-update. For a review prompt, a missed `_inherit` costs some quality. For deciding a blast radius it means an untested flow, which is a different order of problem.



Layer 1: Odoo's own metadata is the knowledge base

This is the part worth internalising, because it removes most of the guesswork. `ir.model.data` attributes every field, view, ACL, rule, and action to the module that created it. So over plain authenticated RPC, with no source access and no shell, the gate can derive:

- The installed module set with versions and authors, and which of those we hold source for

- Every field on a model and **which module added it**, including whether it is stored, computed, or related
- View inheritance chains with per-node module ownership, so you can see who patches whose `xpath` and in what order
- ACLs and record rules, per module
- **Studio customizations**: fields with `state = 'manual'`, and records owned by the `studio_customization` module. This is the set that exists in no repository at all, and it is invisible to any repo-based analysis
- Server actions, automation rules, and crons, per module

```
# attribute every field on the blast-radius models to its owning module
fields = env['ir.model.fields'].search_read(
    [('model', 'in', models)],
    ['name', 'model', 'ttype', 'store', 'compute', 'related', 'state'])

owners = env['ir.model.data'].search_read(
    [('model', '=', 'ir.model.fields'),
     ('res_id', 'in', [f['id'] for f in fields])],
    ['module', 'res_id'])
# -> field id -> owning module, for every field, including third-party ones
```

`ir.model.data` is the path that behaves identically across 17, 18, and 19, which is why the census uses it rather than the convenience columns some versions add. The whole census is a handful of `search_read` calls and takes seconds, which is what makes rebuilding it per run affordable.

The consequence is that you never have to guess what a third-party module does to a model you are changing. If the Emipro theme patches the sale order form, the instance says so. If `muk_web_chatter` overrides a view in your change's inheritance chain, the instance says so, and it says so for the client where it is actually installed rather than for all of them.

THE HONEST LIMIT OF METADATA, AND THE FALLBACK

Metadata tells you what a module *declares*, not what its Python *does*. A third-party module that overrides `write()` in a way that breaks your computed field leaves no trace in `ir.model.data`.

Where we hold the source, layer 2 covers it. Where we do not, there are two fallbacks, in order: read the module's source off the instance filesystem when the client has the `shell` capability, and otherwise **treat unknown-source modules as high-risk consumers whose flows are always selected rather than reasoned about**. When you cannot analyse it, test it. That is a worse outcome than analysis but a much better one than assuming it is inert.

Layer 2: the source map replaces the regex extractor

The impact engine already needs a real AST parse in phase 0. That parser produces a strict superset of what the existing regex extractor produces, so the two consolidate: one extractor, the same JSON shape, richer content.

Method overrides and whether they call `super()`, `@api.depends` chains, `@api.constrains`, raw SQL, `sudo()` use, and outbound HTTP imports all become available.

Worth noting that this improves code review as a side effect, not just the gate. `render_for_prompt` keeps its focus-file scoping and its 30 KB cap and simply has better facts to render, so the reviewer prompt gets more accurate at no extra cost.

Layer 3: the curated overlay

What cannot be derived from anywhere: business invariants, regulatory constraints, the reason a module is fragile, which apps the client does not use. It lives in the *client's own repo* at `qa/knowledge.yml` next to their scenarios, changes by pull request, and is small by design.

`qa/knowledge.yml`

```
client: legacymakermeats

invariants:
- id: INV-01
  text: >
    The kill sheet must fit on a single page. A layout change that pushes to a
    second page is a regulatory problem, not a cosmetic one.
  scope: {models: [mrp.production], reports: [hst_kill_sheet.report_kill_sheet]}
  added_by: hamza
  last_confirmed: 2026-08-12
  review_after: 2027-03-01

danger_zones:
- id: DZ-01
  text: >
    The QuickBooks connector posts to the client's live accounting. Never
    exercise it without a registered stub, in any tier.
  scope: {modules: [pragmatic_quickbooks_connector]}
  review_after: 2026-12-01

expected_values:
  sales_tax_default: 8.25
  fiscal_year_start: "07-01"
  weight_uom: lb

unused_apps: [mrp_repair, website_sale]    # never select these library flows
```

`unused_apps` earns its place immediately: it stops the gate spending six minutes a run testing e-commerce checkout for a client with no website. `expected_values` is what lets shared library flows assert on tax and rounding without every client needing their own copy of the flow.

HOW THIS LAYER STAYS CURRENT, GIVEN THAT NOBODY UPDATES WIKIS

Four mechanisms, none of which is "remember to keep it up to date":

MECHANISM	HOW IT WORKS
Decay	Every entry carries <code>review_after</code> . Past that date it renders as stale in the run bundle and on the client's dashboard page. A stale entry still applies, it just announces that nobody has vouched for it lately
Confirm at the moment of relevance	When a run's blast radius intersects an entry's <code>scope</code> , the bundle surfaces that entry to whoever reviews the run and asks them to confirm or amend it. One click updates <code>last_confirmed</code> . Asking someone about an area while they are already looking at it is the only review prompt that reliably gets answered
Contradiction detection	If an entry names a model, module, or report that the census says no longer exists, it is flagged automatically. This catches the most common form of rot: knowledge about something that was renamed or uninstalled
Pull request review	It lives in the client repo, so it changes through the same review the code does, by the people who just learned the thing being written down

The AI may *propose* overlay entries from a run's findings, presented for acceptance alongside proposed scenarios. It never writes one directly, for the same reason it never decides a verdict.

Layer 4: regression memory

Append-only, written by the gate itself, and the layer that makes the knowledge base get smarter rather than merely larger.

Every confirmed regression stores the **signal fingerprint** of the diff that caused it, the scenario that caught it, the model and method involved, and the fix. When a later diff's signals resemble a stored fingerprint, those scenarios are force-selected even if the blast radius would not have reached them.

```
signal:  mrp.production :: write override
caught:  mrp.unbuild_finished_order
symptom: ValueError: Expected singleton
fix:     override iterated records instead of assuming one
recorded: 2026-08-31, run-8421, task 4471
→ any future write override on mrp.production auto-selects the unbuild scenarios
```

This targets the way Odoo failures actually cluster, which is by model and method rather than by feature. It costs nothing to store, and after twenty or thirty regressions it is selecting scenarios that no static analysis of the diff would have justified.

The same layer accumulates **learned cleanup rules**: which models refused `unlink` in which state, and which reversal worked. So the tier 3 cleanup walker gets better per client over time without anyone writing rules by hand, and residue trends down as a consequence of running rather than as a separate project.

The census doubles as a change feed

The instance fingerprint from section 8 is computed from the census, so watching for baseline validity and watching for knowledge drift are the same operation. Any census difference between runs is reported:

```
Since last night on legacymakermeats_staging:
+ module  hr_attendance_geolocation 18.0.1.0.2  installed, not in any repo we hold
+ field    mrp.production.x_pen_number          Studio, manual, no module owner
~ view     mrp.view_mrp_production_form         now also patched by muk_web_theme
! entry    INV-01 references hst_kill_sheet     still present, review_after passed
```

That feed is worth having on its own. Somebody installing an app on a client's staging instance without telling anyone is currently invisible until it breaks something, and it is a routine cause of the mystery failure that eats an afternoon.

What the humans get: a coverage map

One page per client in the dashboard, and its most useful output is not context for a language model. It is the answer to "which of this client's modules is most exposed right now".

MODULE	SOURCE	MODELS TOUCHED	SCENARIOS	LAST CHANGED	EXPOSURE
<code>hst_mo_carcass_weight</code>	ours	<code>mrp.production</code>	14	2 days ago	COVERED
<code>hst_kill_sheet</code>	ours	<code>mrp.production</code> , <code>report</code>	6	3 weeks ago	COVERED
<code>hst_lot_weight</code>	ours	<code>stock.lot</code> , <code>stock.move.line</code>	0	3 days ago	EXPOSED
<code>pragmatic_quickbooks_connector</code>	vendor	<code>account.move</code> , <code>res.partner</code>	2	unknown	STUB ONLY
<code>muk_web_theme</code>	vendor	views only, 34 patches	0	unknown	VIEW RISK
<code>studio_customization</code>	none	7 manual fields	0	from census	NO SOURCE

The third row is the one that earns the page. A module changed three days ago with zero scenarios covering it is where the next client-reported regression is going to come from, and no amount of prose in a wiki surfaces that as clearly as a sorted column. The bottom three rows are what the instance census adds that a repo-only knowledge base cannot see at all.

Conflicts get their own list on the same page, computed from the census: two modules patching the same `xpath`, two overriding the same method, a custom module whose compute depends on a Studio field that no repository declares.

Cost, and where it lands in the roadmap

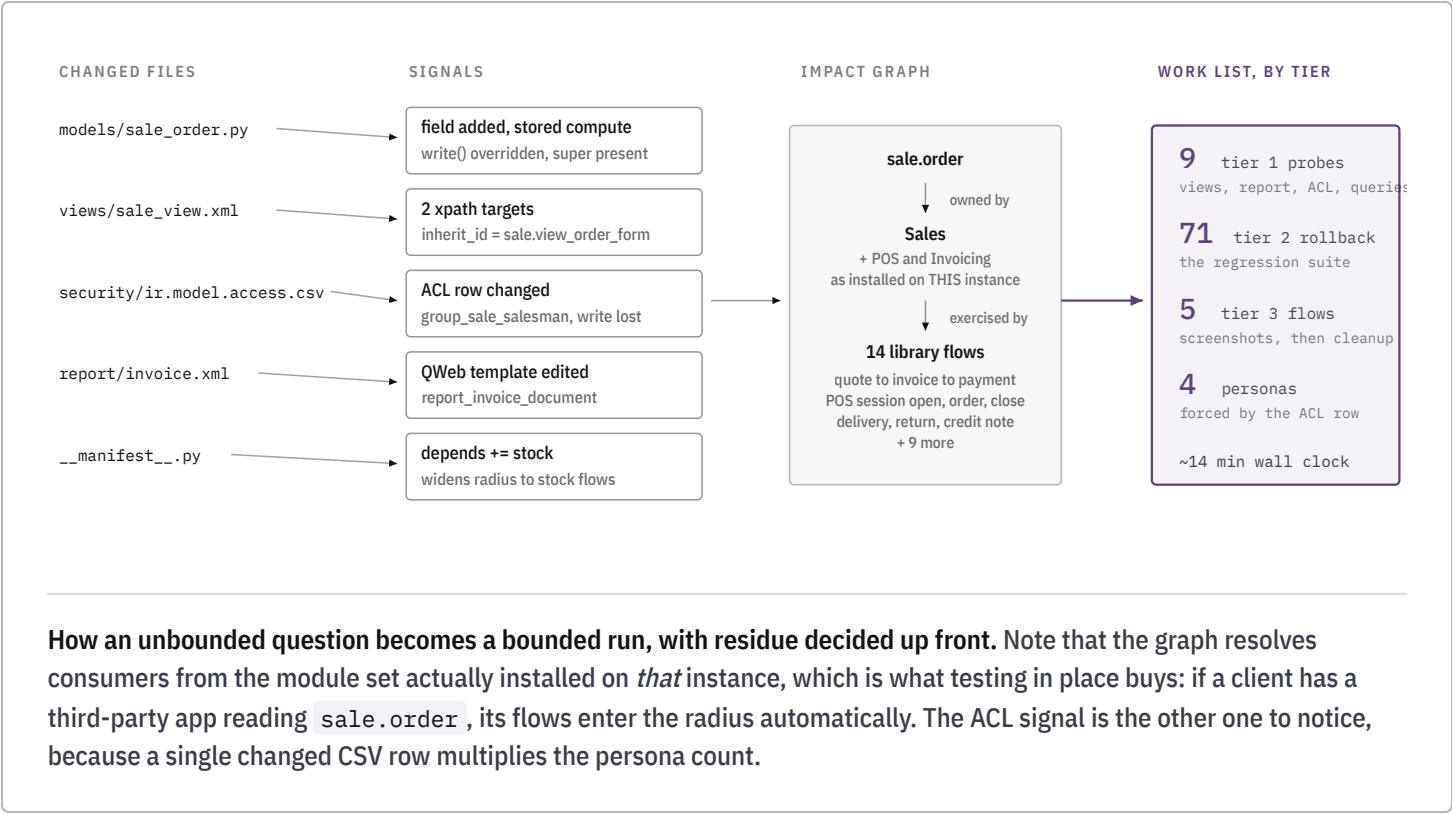
LAYER	BUILD COST	PHASE	ONGOING COST
Instance census	3 to 4 days	0, alongside the hygiene audit, since both are read-only RPC	None. Re-derived per run
Source map	Already in phase 0's impact engine	0	None. Re-derived per run
Curated overlay	1 day for the schema and the decay checks	2	Minutes per entry, prompted at review time
Regression memory	2 days	2, once verdicts exist to record	None. Written by the gate
Coverage page and change feed	3 days	5, with the dashboard merge	None

Roughly two weeks in total, most of it already inside phases that exist for other reasons. The census in particular is worth pulling forward: it is read-only, needs only RPC, and a first run of it across every client tells you today which instances carry modules and Studio fields that exist in no repository you own. That is a useful answer regardless of whether the gate gets built.

10 Blast radius analysis

You asked for "all possible scenarios". That set is infinite, and a tool that chases it produces a suite so slow and flaky the team routes around it. The workable version is risk-based selection: read the diff, work out what it can possibly have broken, and test that exhaustively.

The runner parses changed files into **signals**, maps signals to affected models, maps models to the flows that touch them, and emits a bounded work list with a tier assigned to each item. Python goes through the standard library `ast` module, XML through `lxml`. No regex over source.



Signal table

The core lookup, and the part to get right first. It grows as you learn which changes bite. The tier column is new in this revision: it is what keeps residue predictable before a single scenario runs.

DETECTED SIGNAL	RISK	TIER	WHAT GETS SELECTED
<code>_inherit</code> on a model	BASE	2	All library flows for that model and its installed consumers
New stored computed field	HIGH	1, 2	NULL count over existing rows as a read-only probe, recompute correctness in rollback, plus the <code>@api.depends</code> chain
<code>@api.depends</code> edited	HIGH	2	Write-then-read assertions on every field downstream of the changed dependency
<code>create</code> / <code>write</code> / <code>unlink</code> override	HIGH	2	Full flow set for the model. Runner also asserts a <code>super()</code> call exists on every branch and reports a finding if not
<code>@api.constrains</code> added	HIGH	1, 2	Validation sweep over existing records, because a new constraint can make historical data unsavable. The sweep is read-only, the fix verification is rollback
<code>self.env.cr.execute</code>	HIGH	2	Raw SQL bypasses the ORM. Multi-company and record-rule scenarios forced on, plus a query-count probe
<code>.sudo()</code> added	MED	1, 2	Persona matrix expanded to portal and minimal-rights users to check nothing leaks
View <code>xpath</code> changed or added	MED	1, 3	Resolve every view inheriting the same target in every mode read-only, then render the form in the browser
<code>ir.model.access.csv</code> or <code>ir.rule</code>	HIGH	1, 2	Full persona matrix on every flow touching that model. The highest-value automatic selection in the table
QWeb report template	MED	1	Render for 5 sampled real records, assert no traceback, diff text and page count against baseline. Entirely read-only, which makes it cheap and safe
<code>assets</code> , or any JS or SCSS	MED	3	Browser tier forced on. Console error budget of zero, asset bundle build check
<code>depends</code> added to the manifest	MED	2	Radius widened to the new dependency's flows. Whether the dependency can later be <i>removed</i> cleanly is not checkable here and is reported as an open risk
New module directory	BASE	1, 2	Install and upgrade are exercised by deploying it, and the full flow set runs against it. Uninstall is never tested , because uninstalling on the client's only staging instance is destructive and there is no copy to do it on
Data file with <code>noupdate="1"</code> edited	MED	static	Reported as a finding on its own, since the edit will not apply to existing databases without a migration
Migration script under <code>migrations/</code>	HIGH	static	The gate cannot verify this and says so. Running a migration is a one-way change to the only database there is. Instead the script is read: non-idempotent patterns, unbounded <code>UPDATE</code> s, and <code>WHERE</code> clauses that silently skip rows are reported as findings, and a tier 1 probe counts the rows the script claims to touch. Execution stays a human review with a database backup taken first

CHEAP, AND WORTH DOING IN WEEK ONE

Three rows there are pure static analysis and need no instance access at all: the missing `super()` check, the edited `noupdate` data, and a stored compute added with no migration to populate it. Ship them as a standalone linter in `odoo-dev-loop`'s review step and they start catching real problems before any of the rest exists.

11 The test plan

The gate selects work mechanically and proposes scenarios with a language model. Neither is visible unless you deliberately make it visible. The test plan is the artifact that puts both in plain language, in front of a human, *before* the run executes.

It answers three questions a reviewer actually has. What did you understand the task to be. What workflow will you walk through. What exactly are you going to test, and why that.

Reading the task properly

The task description is the primary input, and real ones are messier than the convention in section 14 assumes. So the interpreter reads more than one field:

SOURCE	WHY IT IS READ
description	The main requirement. An HTML field, so it needs converting to text with list structure and line numbering preserved, because citations point at lines
The acceptance fenced block	Machine-readable criteria where the convention has been followed. Its absence is itself worth reporting
Chatter, mail.message on the task	Requirements arrive as follow-up comments constantly. A plan built only from the description will miss the clarification the project manager posted three days later, which is exactly the requirement the client cares about
Attachments	Named and listed. Where the AI layer has a vision-capable model configured, screenshots and mockups are read too. Where it does not, an attachment-only specification is reported as a gap rather than guessed at
allocated_hours , stage, assignee	Context and scale. A 2-hour task with a 40-step interpretation means the interpretation is wrong

Two practical notes from your task corpus. Descriptions mix languages, so the interpreter must restate in English without silently dropping content it did not parse. And a description that amounts to "see the attached email" must produce a plan that says **insufficient specification** loudly rather than inventing a plausible requirement, because a confident interpretation of a task nobody actually specified is worse than no plan at all.

Provenance on every step, which is the whole point

A reviewer scanning a plan needs to know instantly which steps are machine-selected and safe to skim, and which are model judgment and worth reading closely. So every test step carries a `from` tag:

TAG	MEANS	HOW MUCH SCRUTINY
AC1	Traces to an acceptance criterion in the task	Skim. Check the mapping is sane
desc:L4	Derived from a specific line of the description, quoted alongside	Read. This is where a misreading shows up
signal:ac1-change	Selected deterministically by the blast radius, not from the task at all	Skim. Mechanical, and the reason coverage exceeds what was asked for
flow:sale.confirm_to_invoice	A standing library flow pulled in by the impact graph	Skim
memory:reg-0042	Forced in by regression memory, because a similar diff broke this before	Skim, and be glad
kb:INV-01	Required by a curated knowledge entry for this client	Skim. Confirm the entry is still true if it is stale
ai:new	A scenario the model proposed that does not exist yet	Read closely. This is the only category that needs ratification before it becomes permanent

That single column is what makes the plan a five-minute read rather than a full audit. In a typical run, two or three steps carry `desc:` or `ai:new` and the other eighty are mechanical.

What a plan looks like

Continuing the Legacy Maker Meats task from section 17. Posted to the Odoo task as a chatter note before execution, and carried in the bundle as `plan.json` and `plan.html`.

PART 1 · INTERPRETATION

	WHAT I UNDERSTOOD	FROM
I1	A stored field holding carcass weight in kilograms must exist on the manufacturing order, alongside the existing pounds field	desc:L2 "capture a secondary carcass weight in kg alongside the existing lbs field"
I2	Both weights must appear on the kill sheet report	desc:L3 AC2
I3	When the two weights disagree by more than 2 percent, confirming the MO requires a user in the Production Manager group	desc:L4 AC3 chatter 2026-08-28

PART 2 · WORKFLOW, AS A USER PERFORMS IT

1. A production user opens a manufacturing order for a carcass
2. Enters the carcass weight in pounds, as they do today
3. Enters the secondary weight in kilograms
4. The two agree within 2 percent, so the MO confirms normally
5. On a second MO the two disagree by more than 2 percent, so confirmation is refused and approval is requested
6. A Production Manager approves, and the MO confirms
7. The kill sheet prints, showing both weights on one page

PART 3 · TEST STEPS

STEP		TIER	PERSONA	FROM	COVERS
T01	Create an MO, set weight in lbs, assert the kg field is stored and correct	2	production_user	AC1 desc:L2	AC1
T02	Edit the lbs value, assert kg recomputes	2	production_user	signal:api-depends	AC1
T03	Count existing MOs where the new kg field is empty	1	admin	signal:stored-compute	none
T04	Variance under 2 percent, confirm succeeds with no approval	2	production_user	AC3	AC3
T05	Variance over 2 percent, confirm is refused	2	production_user	AC3 desc:L4	AC3
T06	Variance over 2 percent, confirm succeeds as a Production Manager	2	production_manager	AC3	AC3
T07	A quality user can still write to the MO as they could before	2	quality_user	signal:ac1-change	none
T08	Render the kill sheet for 5 real MOs, assert both weights present and page count still 1	1	production_user	AC2 kb:INV-01	AC2
T09	Unbuild a finished MO	2	production_user	memory:reg-0042	none
T10	Browser walkthrough of the workflow above, 23 screenshots	3	production_user	AC1 AC2 AC3	AC1 AC2 AC3

Three of those rows are worth pointing at, because they are the argument for the whole design. T07 is not in the task anywhere. It exists because a CSV row changed, and it is the step that found a real bug in section 17's walkthrough. T08 asserts a page count because a curated knowledge entry says the kill sheet is legally required to fit one page, which the task never mentions. T09 exists purely because a similar diff broke unbuild once before.

PART 4 · GAPS, ASSUMPTIONS, AND WHAT IS NOT COVERED

The most valuable part of the plan, and the reason to read it before the run rather than after.

ISSUE		WHAT I ASSUMED
AMB-01	"disagree by more than 2 percent" does not say 2 percent <i>of which value</i>	Taken as 2 percent of the pounds value converted to kilograms, treating pounds as the reference. If that is wrong, T04 to T06 are all testing the wrong threshold
AMB-02	Nothing says whether existing manufacturing orders should be backfilled	Taken as not required. T03 will report how many rows are affected either way
GAP-01	Nothing describes the behaviour on an MO already confirmed before this change ships	No step covers it. Flagged rather than invented
GAP-02	AC3 mentions a Production Manager group that did not exist before this task	Assumed the new group in the diff is the one meant. Worth a glance, since the group's other members inherit the permission too

THIS IS THE HIGHEST-VALUE OUTPUT IN THE WHOLE TOOL

AMB-01 is a genuine specification ambiguity. If the reviewer reads it and says "no, 2 percent of the kilogram value", the fix costs five minutes now. Caught after the run, it costs a full test cycle. Caught after the client sees it, it costs a rework and a conversation.

A misread requirement is more expensive than a bug, and it is the one class of problem no amount of test execution finds. Which is why the plan exists as a separate reviewable artifact rather than as a paragraph at the top of the results.

Approval modes

Whether the run waits for a human is a per-client setting, and the default matters:

MODE	BEHAVIOUR	WHEN
advisory	The plan is posted to the task and the run proceeds immediately. You read it whenever you like, and object after the fact	The default for routine work, because blocking every run on a human defeats the automation
approve_first	The run produces the plan, then stops and waits. Verdict <code>AWAITING APPROVAL</code> , no instance writes yet	New clients, high-risk diffs, and while the team is still building trust in the tool
auto	Recommended. <code>approve_first</code> whenever the plan contains an <code>ai:new</code> step or any <code>AMB</code> entry, <code>advisory</code> otherwise	Steady state. Human attention goes exactly where something new or uncertain is happening, and a run reusing only ratified scenarios never waits

The `auto` rule is the one to aim for. In practice a task whose scenarios all already exist produces a fully mechanical plan and runs unattended, while the first task to touch a new module stops and asks. That is the same trust-building shape as enabling blocking per client, applied to attention rather than to gating.

Plan diffs on a re-run

When a bounced task comes back, the plan is regenerated and **diffed against the previous one**. A reviewer who already read the first plan then reads three lines instead of forty:

```
Plan diff, run-8433 vs run-8421
= I1 I2 I3 unchanged
+ T11 migration populates kg on existing MOs      from AMB-02 resolved, desc:chatter
~ T07 now expects quality_user CAN write          was expected write to be refused
- AMB-01 resolved: 2% of the pounds value, confirmed by hamza 2026-09-01
```

If the interpretation itself changed between runs without the task changing, that is worth knowing and the diff makes it obvious.

Run the interpreter at assignment time as well

The same component has a second, arguably higher-value placement. Run parts 1, 2, and 4 when a task is *assigned*, before any code is written, and post them to the task.

A developer then starts from a restated requirement and an explicit ambiguity list. AMB-01 gets resolved before three days of work is built on the wrong threshold, rather than after. Nothing about the implementation is needed to produce it, since parts 1, 2, and 4 come from the task text alone, so this costs one extra AI call at assignment and no infrastructure.

It also improves the tasks themselves over time. A team that sees its own descriptions restated back at it, with the ambiguities listed, writes better descriptions within a month. That is a real side effect and it is worth having even if the gate is never enabled for blocking.

WHAT THE PLAN DELIBERATELY DOES NOT DO

Selection stays deterministic. The impact graph picks the scenarios, and the model writes prose about them, proposes new ones for ratification, and restates the task. It does not choose what runs, and the plan never influences the verdict.

The distinction shows up in the `from` column, which is why that column is not decoration. Every step is either traceable to a mechanical rule, to a line of the task, or to a proposal explicitly marked as needing a human. There is no fourth category, and if a step ever appears without provenance that is a bug in the generator.

12 The scenario format

One YAML file per scenario, declarative, checked into the client repo under `qa/scenarios/`. The same file drives either the rollback tier or the browser tier, which is the point: a business flow is described once, run fast and reversibly in the regression suite, and re-run in the browser when it needs to produce screenshots.

`qa/scenarios/sale/line_discount.yml`

```

id: sale.line_discount
title: A line discount reduces the order total and carries to the invoice
tags: [sale, regression, ratified]
versions: ["17.0", "18.0", "19.0"]
personas: [sales_user, sales_manager]
tier: 2 # rollback. required key, no default
drift: immune # immune | data_dependent
covers: [AC1, AC2]

fixtures: # created inside the transaction, never reused
  partner: {model: res.partner, pick: {customer_rank: {">": 0}}, limit: 1}
  product: {model: product.product, new: {name: "QA-{{run}} Widget",
    list_price: 100.0, type: consu, taxes_id: []}}

steps:
  - create:
    model: sale.order
    values:
      partner_id: $partner
      order_line: [[0, 0, {product_id: $product, product_uom_qty: 10}]]
    as: order
  - assert: {expr: "$order.amount_total", equals: 1000.0}
  - write: {on: "$order.order_line[0]", values: {discount: 10.0}}
  - assert: {expr: "$order.amount_total", equals: 900.0, because: "10% off 1000"}
  - call: {on: $order, method: action_confirm}
  - assert: {expr: "$order.state", equals: sale}
  - call: {on: $order, method: _create_invoices, as: invoice}
  - assert: {expr: "$invoice.amount_untaxed", equals: 900.0}
  - assert.no_traceback: true

```

The whole file travels as one `run_scenario` call and comes back rolled back. An evidence scenario is the same grammar at tier 3, with UI verbs and explicit capture points. Selectors are *logical*, written as `@field:name` or `@button:method`, and resolved to real CSS by the version adapter, which is what stops a v19 upgrade from invalidating every scenario you own.

qa/scenarios/sale/line_discount_evidence.yml


```

id: sale.line_discount_evidence
extends: sale.line_discount      # inherits fixtures and assertions
tier: 3                          # commits, so cleanup is owed
personas: [sales_user]
cleanup: reverse                 # reverse | keep_as_evidence

steps:
- ui.goto: {action: "sale.action_orders"}
  shot: "Sales orders, before the change"
- ui.click: {sel: "@list:new"}
- ui.fill: {sel: "@field:partner_id", value: $partner.name, label: "Customer"}
- ui.fill_row:
  field: order_line
  values: {product_id: $product.name, product_uom_qty: 10}
  shot: "Order line added at list price"
- ui.fill: {sel: "@field:order_line.discount", value: 10, label: "Discount (%)"}
- ui.save
  shot:
    caption: "Total recalculated after a 10% line discount"
    highlight: "@field:amount_total"
    readback: [amount_untaxed, amount_total, "order_line.price_subtotal"]
- assert.ui: {sel: "@field:amount_total", reads: "900.00"}
- ui.click: {sel: "@button:action_confirm"}
  shot: "Order confirmed, delivery and invoice smart buttons present"

```

Four details that matter more than they look

TIER IS REQUIRED AND HAS NO DEFAULT

An author has to state what the scenario may do to the instance. There is no accidental path to committing on a client's staging database, and reviewing a scenario means reviewing its residue profile. This is the schema-level expression of the whole revision.

READBACK, NOT JUST INPUT

After every save the runner reads the fields back out over RPC and records both what was typed and what the database ended up holding. Odoo `onchange` handlers routinely change a value after you enter it, and a screenshot alone cannot distinguish "the user typed 900" from "the system computed 900". The readback column is what makes the evidence bundle actually evidential.

BECAUSE, ON EVERY ASSERTION

The optional `because` string is what appears in the report when the assertion fails. Without it a reviewer gets `expected 900.0, got 1000.0` and has to go read the YAML. With it they get a sentence. Make it required in review even though the schema allows omitting it.

NO WALL-CLOCK DEPENDENCE

Scenarios never assert on "today". A run pins `TZ`, and a scenario needing a date uses an explicit `as_of` value carried into the Odoo context. A scenario asserting on today's date passes for a month and then fails on the first of the next one, and that single class of flake will cost you the team's trust faster than anything else here.

13 The flow library

This is the reusable asset. Scenarios in a client repo are specific to that client. The flow library is the shared set of standard Odoo business processes, maintained once per version in the gate's own repo, and it is what lets a new client get meaningful regression coverage on day one with no scenario authoring at all.

APP	FLows TO COVER FIRST	FLows	NOTES
Sales	Quote, confirm, deliver, invoice, register payment, credit note, discount, multi-currency, down payment	14	Highest reuse across your client base
Inventory	Receipt, internal transfer, delivery, return, scrap, lot and serial assignment, reservation, backorder	16	Lot and serial flows matter for the meat and food clients
Purchase	RFQ, confirm, receive, bill, three-way match, partial receipt	9	
Accounting	Invoice, payment, reconcile, tax computation, journal entry, lock date, fiscal position	12	Tax assertions need per-client expectations, so those live in the client repo
Manufacturing	BoM, MO confirm, component consumption, byproducts, done, unbuild	10	Carcass weight and kill-sheet work sits here
Point of Sale	Open session, order, discount, payment, refund, close session and post	8	Tier 3 only, and the most expensive to maintain. See section 22
HR	Employee, contract, attendance, leave request and approval, payslip	11	Attendance device integrations must be stubbed
Website and portal	Portal login, document visibility, e-commerce checkout	7	Doubles as the portal-persona security check

Roughly 85 to 90 flows to reach useful coverage of the apps your clients actually run. That is the biggest line item in the roadmap. Three things make it tractable:

- **Do not write them all up front.** Write a flow the first time a task's blast radius needs it. After ten or fifteen tasks the common ones exist, and the library grows along the curve of what you actually touch.
- **Fixtures resolve by query, never by external id.** The `pick:` clause matters more now than it did in revision 1: an XML id that exists in a demo database will not exist on a client's staging instance, and hard-coding one is the most common reason a shared flow fails to port between clients.
- **Most flows are tier 2**, so they cost nothing in residue and can be written freely. Only the handful that need screenshots get promoted to tier 3, and promoting one is a deliberate review decision.

14 The evidence bundle

This is the deliverable, and the part a project manager or client will judge the tool by. Everything else exists to produce it.

One directory per run, uploaded and served by the control plane as a self-contained HTML report with a print stylesheet, so it exports to PDF for a client.

```
runs/2026-08-31/run-8421/
  plan.html          # the test plan, as posted to the task before the run
  plan.json          # interpretation, workflow, steps with provenance, gaps
  report.html        # self-contained, print-ready
  summary.json        # machine-readable, feeds the dashboard
  verdict.json        # gate decision + findings[]
  preflight.json      # every instance-contract check and its outcome
  fingerprint.json    # instance state, and the baseline it was compared to
  residue.json        # what the run left behind, per record, and why
  regression-diff.json # baseline vs candidate, per scenario
shots/
  001-sales-orders-before.png
  002-order-line-added.png
  003-total-recalculated.png      # + .dom.html alongside
ledger.csv                      # every value entered, per step
logs/
  guard-events.json      # blocked mail, blocked HTTP, attempted commits
  console-errors.json
  queries.json           # per-scenario query counts vs baseline
traces/
  flow-pos-close.zip      # Playwright trace, failures only
```

Four of those files make the run auditable rather than merely reported. `plan.json` is what the gate said it would do, written before it did it. `preflight.json` proves the instance was safe to touch, `fingerprint.json` proves the baseline comparison was valid, and `residue.json` accounts for everything the run added. Between them, a reviewer can check the intent, the safety, the validity, and the footprint of a run without reading a single screenshot.

What a single screenshot record carries

A bare PNG is not evidence. Each capture point in the report shows:

FIELD	EXAMPLE	WHY IT IS THERE
Step number and persona	003 · sales_user	Order is the whole point, and which user matters as much as which screen
Caption	Total recalculated after a 10% line discount	Written by the scenario author, not generated at report time
Screenshot	Full page, target element outlined in the accent colour	The outline is burned into the PNG so it survives being pasted into an email. Captions stay in the report HTML so they remain searchable
Record identity	sale.order#4471 · S00312	The reviewer can open the record on the same staging instance and look for themselves. On in-place testing this link actually resolves, which it did not in revision 1
Value ledger slice	See below	What was typed against what was stored
Assertions at this step	PASS amount_total reads 900.00	Ties the picture to a machine-checked claim
Console and network	0 errors, 0 responses ≥ 500	Silent breakage is invisible in a screenshot
Cleanup outcome	REMOVED or REVERSED	New in this revision. Every record shown in a screenshot says what happened to it afterwards

THE VALUE LEDGER

FIELD LABEL	TECHNICAL NAME	ENTERED	STORED AFTER SAVE	
Customer	partner_id	Deco Addict	Deco Addict (#14)	MATCH
Product	order_line[0].product_id	QA-8421 Widget	QA-8421 Widget	MATCH
Quantity	order_line[0].product_uom_qty	10	10.0	MATCH
Unit price	order_line[0].price_unit	not entered	100.0	COMPUTED
Discount (%)	order_line[0].discount	10	10.0	MATCH
Untaxed amount	amount_untaxed	not entered	900.0	COMPUTED

The

COMPUTED

 rows are the ones a reviewer should read closely, because that is where Odoo did something on its own. A row reading

CHANGED

, meaning entered and stored disagree, is surfaced at the top of the report whether or not any assertion failed.

The residue report

New, and it is what earns the right to write to a client's staging instance at all. Every record the run created, and what became of it:

MODEL	RECORD	CREATED AT STEP	OUTCOME	WHY
product.product	QA-8421 Widget	fixture	REMOVED	Unlinked cleanly
sale.order	S00312	004	REMOVED	Cancelled, then unlinked
stock.picking	WH/OUT/00891	009	REMOVED	Cancelled, then unlinked
account.move	INV/2026/00447	012	REVERSED	Posted entries cannot be deleted in Odoo. Reversal INV/2026/00448 also remains
ir.sequence	sale.order, 3 values	various	GAP	Postgres sequences do not roll back. Numbering gap only

Read that table as the honest answer to "does it affect existing data". No existing record appears in it, because tier 3 is forbidden from touching one. What remains on the instance is two accounting entries that cancel each other out and three consumed order numbers, all named and explained. A weekly sweep reconciles the registry against the instance and reports anything cleanup missed.

The summary paragraph

One AI-written paragraph at the top, for a reader who will not scroll. Generated from `verdict.json` and the task description, constrained hard: state the verdict, state what was demonstrated, state what broke, name no file paths. Uses the PMO dashboard's existing multi-provider AI layer with its fallback chain, so it costs nothing new to operate.

KEEP THE AI OUT OF THE VERDICT

The verdict is computed from assertion results and the baseline diff, deterministically. The AI writes prose about a decision already made, and proposes scenarios a human ratifies. It never decides pass or fail, and it never decides whether an instance is safe to touch. The moment a language model sits in the decision path, every argument about a result becomes unresolvable and the gate stops being a gate.

15 Verdicts and gate policy

VERDICT	MEANS	GATE	ODOO STAGE
PASS	Every acceptance criterion demonstrated, no new failures against baseline	Open	Moves to PM Review, bundle in chatter
REGRESSION	A scenario passed at baseline and failed at head, with fingerprints agreeing	Blocked	Back to In Progress, developer assigned, diff in chatter
UNMET	Suite is green but an acceptance criterion has no passing assertion covering it	Blocked	Back to In Progress. Usually means the scenario was never written
AWAITING APPROVAL	The test plan is generated and the run is paused for a human, under <code>approve_first</code> or the <code>auto</code> rule	Paused, nothing written to the instance yet	Stays in QA Gate. Plan posted to chatter with an approve action. See section 11
PAUSED	The run stopped at a checkpoint and released the instance, either on request or because the runner could not continue	Nothing. Not a result	Stays in QA Gate with the checkpoint summary and interim residue in chatter, so a paused run reads as clearly as a finished one. New in revision 3. See section 7
NEEDS RE-BASELINE	New failures, but the instance fingerprint moved since the baseline, or moved during a pause	Warns, does not block	PM Review with a banner. Runner queues a fresh baseline
INSTANCE UNSAFE	The pre-flight audit refused. Crons live, mail server active, sentinel missing	Blocked, no blame	No stage change. Alerts the lead and names the failed check. New in this revision
RESIDUE	Suite green, but cleanup could not remove something it should have	Warns	PM Review with the residue table surfaced. Repeated occurrences on the same model are a bug in the cleanup rules, not in the task
FLAKY	Failed then passed on retry	Open, warns	PM Review with a warning. Auto-quarantines after 3 in 10
INCONCLUSIVE	Infrastructure failed: deploy, lock timeout, instance unreachable	Blocked, no blame	No stage change, alerts the lead rather than the developer
PRE-EXISTING	Failed at baseline and at head	Open	Separate backlog, never blocks the current task

The no-blame rows are what make the gate survivable. A tool that blames a developer for a client's hosting provider re-enabling crons, or for a bug that predates their branch, gets switched off, and correctly so.

Coverage of acceptance criteria

UNMET needs the task's acceptance criteria to be machine-readable, which means a small convention in how tasks are written. In the Odoo task description, criteria go in a fenced block:

```
```acceptance
- AC1: A line discount percentage reduces the order total
- AC2: The discount carries to the generated invoice
- AC3: Only Sales Managers may enter a discount above 20%
```
```

Each ratified scenario declares which criteria it covers via `covers: [AC1, AC2]`. Any criterion with no passing assertion is **UNMET**. This convention is worth adopting whether or not the gate ever gets built, and AC3 there is exactly the kind of criterion only a persona matrix can verify.

DO NOT TURN THE GATE ON AS A BLOCKER ON DAY ONE

Run it in advisory mode for the first four to six weeks: it produces bundles and comments on tasks, and blocks nothing. Use that window to measure the false-positive rate per client. Enable blocking per client only once its suite has been green across ten consecutive unrelated tasks.

In this revision there is a second reason to wait, beyond credibility. Advisory mode is also how you find out which client staging instances are actually neutralized, which have crons quietly running, and which have a live mail server nobody remembered. Expect that audit to fail on more instances than you would guess, and expect fixing those to be genuinely useful work in its own right.

16 Version adapters

Supporting 17, 18, 19, and whatever follows is not about conditionals scattered through the runner. It is one directory of profiles, and everything version-dependent is forced through it.

```
profiles/
  17.0.yml    18.0.yml    19.0.yml
  |— selectors: logical name -> CSS, for the browser tier
  |— flows:    per-version step overrides for library flows
  |— probe_api: savepoint, flush, and invalidate call names
  |— audit:    where crons, mail servers, and providers live per version
  |— quirks:   known-failing patterns, with links
```

| WHAT DIFFERS BETWEEN VERSIONS | WHERE IT LIVES | EXAMPLE |
|-------------------------------|-------------------------|---|
| Front-end selectors | <code>selectors:</code> | Every OWL release moves class names on list rows, form status, and dialog buttons. The largest and most churn-prone section |
| Probe internals | <code>probe_api:</code> | Cursor savepoint helpers and environment flush and invalidate methods were renamed across recent versions. The probe module is version-branched here and nowhere else |
| Audit target names | <code>audit:</code> | <code>payment_provider</code> was <code>payment_acquirer</code> at 16 and earlier. The pre-flight audit reads model names from the profile rather than hard-coding them |
| Flow step shape | <code>flows:</code> | POS changed substantially at 18, so the session open, order, and close flow needs its own step list per version |
| Cleanup rules | <code>quirks:</code> | Which models refuse <code>unlink</code> in which state, which is exactly what the tier 3 cleanup walker needs to know |

Bringing up a new version

This is the process that decides whether the tool is alive in three years, so make it a documented drill rather than an improvisation:

1. Copy the previous profile and bump it. Run the flow library against a vanilla Odoo install of the new version, locally, with demo data. Nothing client-specific yet, and no client instance involved.
2. Every failure is either a selector to remap or a genuine flow change. Fix the profile until the library is green. Budget two to four days, mostly selectors.
3. Point it at the first client staging instance running the new version. Failures here are the real upgrade findings, and this step is itself a saleable service. See `UC-08`.

Note that step 1 still uses a local throwaway Odoo. Testing in place is the right model for verifying client work, and the wrong model for developing the gate itself. Keep a plain local Odoo checkout per version, each with its own virtualenv and its own throwaway database, as the gate's own development fixture. No containers are needed for this and none should be introduced: the fixture exists so the flow library can be exercised against a vanilla install, and a source checkout does that with less machinery than anything else.

17 End-to-end walkthrough

A real-shaped task, from the developer finishing it to the project manager reading the verdict. Timings are the target budget, not a guess after the fact: the run has to land inside 20 minutes or developers stop waiting for it and start ignoring it.

THE TASK

Client: Legacy Maker Meats · **Odoo 18** · **Module:** `hst_mo_carcass_weight` · **Hosting:** Cloudpepper · **Capabilities:** `rpc` `browser` `shell`

Capture a secondary carcass weight in kilograms alongside the existing pounds field on the manufacturing order, show both on the kill sheet report, and require a Production Manager to approve when the two disagree by more than 2 percent.

1 Developer finishes and pushes

T+0

Work happens as it does today, inside `odoo-dev-loop`. The developer hits *Run code review*, the local AI review passes, and they push their task branch. This client is on `branch_mode: per_task`, so the branch carries only this task.

2 Gate enqueued, per-client lock acquired

T+0

The task moves to a new **QA Gate** stage. The control plane enqueues the run and takes the lock on Legacy Maker Meats' staging instance. Nobody else's run can touch it until this one finishes, and the dashboard shows who holds the lock.

3 Pre-flight audit

T+40s

The runner calls `audit()` over RPC. The sentinel is present and names the right client. Database name matches `*_staging`. Payment providers in test mode. But **two `ir.cron` records are active**, re-enabled during a hosting platform update three weeks ago.

Verdict **INSTANCE UNSAFE**. The run stops before touching anything, the lead is alerted with the exact failed check, and the task does not move. Somebody disables the crons, and the run is retried. *This is the system working correctly*, and in the first month it will happen more often than you expect.

4

Blast radius computed

T+1m

Diff parsed. Signals: a stored compute `carcass_weight_kg` on `mrp.production` with `@api.depends` on the pounds field, a `write` override with `super()` present, one QWeb template edit on the kill sheet, one new row in `ir.model.access.csv` for a new `group_production_manager`.

Selected: 9 tier 1 probes, 71 tier 2 scenarios, 5 tier 3 flows, 4 personas because of the ACL row. Two static findings raised before anything runs: the stored compute has **no migration script to populate existing rows**, and the manifest declares no integration stub for a module that imports `requests`.

There is no migration script in the diff, which is itself the first finding. Had there been one, the gate would have read it and reported on it rather than run it: there is one database, and executing a migration against it is the one-way change being tested. The bundle says that plainly instead of implying a confidence it does not have.

5

Test plan posted, and held for approval

T+2m

The interpretation, a seven-step workflow, ten test steps with provenance tags, and two ambiguities go to the task as a chatter note. Because the plan contains an `AMB` entry, the `auto` rule holds the run rather than proceeding.

The lead reads it and confirms that the 2 percent threshold means 2 percent of the pounds value, then approves. *Under two minutes of human time, and it stops steps T04 to T06 from testing the wrong number for the next three weeks.* Section 11 shows this plan in full.

6

Branch deployed to staging

T+4m

Over the `shell` capability: fetch the branch, checkout, upgrade the module, restart. Two minutes and change. On a client without `shell` this step goes through the hosting panel's deploy mechanism instead, which is the one genuinely awkward part of the design and the one most worth automating per host.

A fingerprint is taken immediately after the upgrade and compared to last night's baseline. The module hash differs only by the module under test, everything else matches, so the comparison stands.

7

Tier 1, read-only probes

T+5m

Every view inheriting the MO form resolves and validates. The kill sheet renders for 5 sampled real manufacturing orders with no traceback, the kg column present, page count unchanged. And the NULL sweep confirms what the static analysis predicted: **4,182 existing MOs have an empty `carcass_weight_kg`**, because there is no migration. Reported with the exact count, which is far more persuasive than a warning about a missing file.

Tier 2, the regression suite

T+11m

71 scenarios across 4 personas, each one a single `run_scenario` call executed and rolled back. **Two failures.**

The first is the one that matters: `mrp.unbuild_finished_order` passed at baseline and fails at head with `ValueError: Expected singleton`, raised from the new `write` override which assumed a single record. That is a **REGRESSION**, and it is exactly what a developer clicking the happy path on staging would never hit.

The second, `account.tax_rounding_multi_line`, also failed at baseline. **PRE-EXISTING**, filed to the backlog, does not block this task.

Guard events: one scenario attempted an outbound HTTP call to the QuickBooks connector's endpoint. Blocked, and reported as a finding, since that module had no stub registered.

Persona matrix

T+13m

Still tier 2, so still reversible. The ACL row is validated by replaying the MO confirm flow as each persona. A stock user is correctly blocked from approving a 2 percent variance. But a *quality* user, never mentioned in the task, is now also blocked from a screen they previously used, because the new ACL row narrowed write access on the model rather than adding a rule alongside it. Reported with the exact persona and screen.

A developer needs staging, so the run pauses

T+14m • held 38m

Another developer clicks *I need staging* on the client's page rather than trying to find out who started the run. The request queues, and the run pauses at the next checkpoint 3 seconds later, mid tier 2, with 44 of 71 scenarios complete. Mode **release**, so the task branch stays deployed and the instance is usable immediately.

Thirty-eight minutes later the requester marks themselves done and the run resumes. The pre-flight audit passes again, head is still the deployed commit, and the fingerprint has moved only in row counts and within tolerance, so **all 44 completed scenarios are kept** and execution picks up at number 45. Timings below are run time and exclude the pause.

Tier 3, evidence

T+19m

5 flows in Chromium against the real staging URL, 23 ordered screenshots, every field entry captured with the value typed and the value read back after save. Every record created is logged in the registry, and a write to any record not in the registry would have aborted the flow. Cleanup runs in reverse: the QA products and MOs unlink cleanly, one posted journal entry is reversed and both entries reported as residue, three sequence values consumed. All five rows appear in `residue.json` and in the report.

.2

Bundle and verdict

T+21m

Verdict **REGRESSION**, computed deterministically. The AI writes the summary paragraph. The lock releases. The task moves back to **In Progress** assigned to the developer, with the regression diff, the 4,182-row NULL count, the quality-user finding, and the unstubbed integration all in chatter. Screenshots are kept despite the failure, because the developer needs them to see what the change did do.

.3

Fix and re-run

next morning, T+13m

The developer fixes the singleton assumption, adds the migration script, widens the ACL rather than narrowing it, and registers a stub for the connector. The re-run reuses the same baseline, which is still fingerprint-valid, and runs the identical selection. Verdict **PASS**, task moves to PM Review with a 23-screenshot walkthrough and a residue table showing two reversal entries and nothing else.

The project manager reads one paragraph and six screenshots instead of clicking through staging for 40 minutes. The `mrp.unbuild` scenario that caught the regression is now permanently part of Legacy Maker Meats' suite and will guard that path for every future task.

The last sentence is the compounding return. The gate's value on task one is a screenshot bundle. Its value on task fifty is a suite that has caught nine regressions nobody would have found before a client did.

18 Use case catalog

What the gate is asked to do, in rough order of how often. The first four are the daily bread. Each carries the tier it needs. Three entries that revision 2 automated are now marked **NOT AUTOMATABLE**: they needed a throwaway copy of the database, and on a managed host there is none. They are kept in the catalog rather than deleted from it, because the work still has to happen and somebody has to know it is now a human's job.

UC-01 Verify a completed task

DAILY

TIER 1, 2, 3

TRIGGER

Developer pushes after code review passes

RUNS

Pre-flight audit, blast-radius selection, temporal baseline comparison, evidence flows for each acceptance criterion

DELIVERS

Verdict, ordered screenshot walkthrough with the value ledger, residue table, attached to the Odoo task

CATCHES

The base case. Task not actually complete, criterion demonstrated only in the developer's head, unrelated flow broken

UC-02 Shared-model regression

HIGHEST VALUE

TIER 2

TRIGGER

Diff touches a model that more than one *installed* app reads

RUNS

Flows for every consumer of the model present on that instance, not just the app under change

CATCHES

A field or override added to `sale.order.line` for a reporting need that breaks POS session close. A `write` override that assumes a singleton. An `_order` change that breaks a downstream report's grouping

WHY NOW

Materially better in this revision. The consumer list comes from the instance's real module set, so a third-party app installed through the hosting panel is included automatically. A stack we assembled ourselves would have missed it entirely

UC-03 Access-rights change

HIGHEST VALUE

TIER 1, 2

TRIGGER

Any change to `ir.model.access.csv`, `ir.rule`, or a group definition

RUNS

Full persona matrix over every flow touching the affected models, including a portal user. Entirely reversible, so it is cheap to run broadly

CATCHES

Permission narrowed for one group and accidentally removed from another. A record rule leaking another company's records. A screen that silently empties for a role nobody tested. **Invisible to an admin session, which is how everyone tests today**

UC-04 View inheritance break

TIER 1, 3

TRIGGER

Changed `xpath`, or a nightly run after the hosting provider applies an Odoo point release

RUNS

Resolve every view inheriting the same target in every mode read-only, then render the form in a browser

CATCHES

An `xpath` that stops matching after an upstream change, which in Odoo means the whole view fails to load rather than degrading gracefully. Two custom modules fighting over the same insertion point, which only shows up with both actually installed

UC-05 Stored compute or field migration

TIER 1, 2

TRIGGER

New or changed stored computed field, or a changed `@api.depends`

RUNS

NULL count over the client's real existing rows as a read-only probe, recompute correctness on a sample in rollback, full downstream dependency chain

CATCHES

The classic: works perfectly on new records, leaves years of historical rows empty, nobody notices until a year-end report. **The row count comes from their actual data**, so the finding is "4,182 records are empty" rather than "you may have forgotten a migration"

UC-06 Report and PDF change

TIER 1

TRIGGER

QWeb template or report action changed

RUNS

Render for 5 sampled real records per persona, text-diff and page-count-diff against baseline, plus an image diff at low tolerance. Completely read-only

CATCHES

A traceback on a record shape the developer did not have. Content silently dropped. A layout pushing to a second page and breaking a pre-printed form. This is the cheapest high-value check in the whole catalog, because it needs no writes at all

UC-07 New module install

TIER 1, 2

UNINSTALL NOT COVERED

TRIGGER

A new module directory appears in the diff

RUNS

Install and upgrade happen anyway as part of deploying the branch, so both are exercised and a failure is caught immediately. The full flow set then runs against the new module

CATCHES

Install that only works on a clean database. A missing dependency that happens to be installed on the developer's machine. A module whose upgrade step throws on real data

NOT COVERED

Uninstall, and this is a genuine gap. Uninstalling on the client's only staging instance is destructive and irreversible, and there is no copy to do it on. Orphan columns, broken views, and dangling ACL rows left by a bad uninstall stay undetected until someone actually uninstalls. The mitigation is a static check that the module declares its removal hooks, which is much weaker than testing it

UC-08 Version upgrade verification

SELLABLE

NO LONGER A DRY RUN

TIER 1, 2, 3

TRIGGER

Manual, when a client's 17 to 18 or 18 to 19 move is being planned or has just been performed by the host

RUNS

The client's full suite against the upgraded staging instance, diffed against the last baseline taken on the old version. Every custom module, every ratified scenario, every library flow

DELIVERS

A findings list that becomes the remediation plan, and the evidence that the upgrade is safe to repeat on production

WHAT CHANGED

This is now verification after the fact, not a dry run before it. A dry run needed two databases on two versions; there is one. So the sequence becomes: the host upgrades staging, and the gate tells you within twenty minutes exactly what broke. Still sellable, still far better than a manual click-through, but it no longer produces the quote *before* the client commits to the work

MITIGATION

Run the flow library against a vanilla install of the target version first, per section 16's bring-up drill. That surfaces the version's own breaking changes without touching a client instance, and it is the cheapest half of the old dry run

UC-09 Data migration script review

NOT AUTOMATABLE

STATIC + TIER 1

TRIGGER

A file appears under `migrations/`

RUNS

Static analysis of the script, plus a read-only probe that counts the rows it claims to touch on the client's real data. No execution

CATCHES

Non-idempotent patterns, unbounded `UPDATE` s, `WHERE` clauses that silently skip rows, and a row count that disagrees with what the developer expected. Enough to catch the common mistakes, not enough to prove the script correct

NOT COVERED

The thing that actually mattered. Before and after row counts, checksums, and the idempotency check by running the script twice all require a database you can throw away. Running a migration against the one staging database is the one-way change being tested, so the gate refuses

WHAT REPLACES IT

Human review with a host-taken backup first, and the gate's static findings attached to the task so the reviewer knows where to look. **This remains the most dangerous category of change in Odoo work, and revision 3 does not fix it.** Worth saying out loud when scoping work that involves one

UC-10 Performance guard

TIER 1, 2

TRIGGER

Every run, automatically, on the flows in the blast radius

RUNS

Query-count probes in-process via the probe module, plus wall-clock timing on list and form loads, both diffed against baseline with a tolerance

CATCHES

N+1 queries introduced by a compute inside a loop. A search with no index behind it. A newly slow list view

WHY NOW

Better in this revision. Measurements come from the client's real data volumes on their real hardware, rather than from a machine of ours, so the numbers actually predict production behaviour. This is the guard for the class of problem that surfaces as a client saying the system got slow

UC-11 Multi-company and multi-currency isolation

TIER 2

TRIGGER

The instance has more than one active company, or raw SQL appears in the diff

RUNS

Key flows replayed per company and with several companies active at once, plus a foreign-currency variant

CATCHES

A missing `company_id` on a new record. Raw SQL ignoring record rules and returning another company's data. Currency rounding applied at the wrong point

UC-12 Point of Sale and heavy-JS flows

HARDEST

TIER 3

TRIGGER

Diff touches POS, or any JS or SCSS asset

RUNS

Browser tier only: open session, order with a discount, pay, refund, close and post. Console error budget of zero

CATCHES

OWL errors leaving the screen half-rendered. A patched POS model that breaks on session close. Offline queue behaviour

NOTE

Also the worst residue profile in the catalog: a closed POS session posts entries that cannot be deleted. Either run POS flows in a QA company, or accept and report the residue. **Expect the lowest automation ceiling here and budget extra time**

UC-13 External integration change

TIER 2

TRIGGER

Diff touches a module with an outbound integration: the QuickBooks connector, an attendance device sync, a receipt printer bridge

RUNS

Against a registered stub. The in-transaction HTTP guard raises on anything unstubbed, so a missed stub fails loudly inside the run

CATCHES

Payload shape changes, error handling, retry behaviour. Also enforces that the module registers a stub at all, which is the structural protection against a staging run reaching a live integration

UC-14 Client acceptance pack

SELLABLE

TIER 3

TRIGGER

Project manager requests it, typically at a sprint or milestone boundary

RUNS

Evidence flows only, for every task in the milestone

DELIVERS

One PDF per milestone: each task, its acceptance criteria, and the screenshots proving each one

WHY NOW

Simpler in this revision. Screenshots come from the client's own staging instance showing their own data, so there is no anonymization question at all and the record links a reviewer sees actually resolve. Storage stays per-client access controlled, and a bundle is never published to a shared URL

UC-15 Nightly drift detection

TIER 1, 2

TRIGGER

Cron, per client, overnight

RUNS

Full suite plus fingerprint against the staging instance as deployed. Doubles as the baseline for the next day's task runs

CATCHES

Breakage from an Odoo point release or a hosting platform update, found by you rather than reported by the client. And it is what makes daytime runs fast, so it pays for itself twice

UC-16 Staging instance hygiene audit

NEW, AND DO IT FIRST

AUDIT ONLY

TRIGGER

On demand, and before onboarding any client onto the gate

RUNS

The pre-flight audit alone, across every client staging instance, writing nothing

DELIVERS

A table of which instances have live crons, active mail servers, payment providers out of test mode, or production URLs configured

WHY IT MATTERS

This is worth building before anything else, and it has value even if the gate is never finished. Right now nobody knows which client staging instances could email real customers. The audit is two days of work, needs only RPC, and the answer is almost certainly worse than anyone expects

19 Repo layout and stack

Standalone from day one, structured so the control plane lifts into hst-pmo-dashboard without being rewritten.

```
odoo-qa-gate/
├── api/                                # TypeScript, Express. Lifts into the dashboard as-is
│   ├── src/routes/                    # runs, reports, runner endpoints
│   ├── src/services/
│   │   ├── queue.service.ts          # per-client lock, one run at a time per instance
│   │   ├── verdict.service.ts        # deterministic, no AI
│   │   └── summary.service.ts        # AI prose only, via the dashboard's provider chain
│   └── src/db/                        # runs, findings, residue, fingerprints, quarantine
├── probe/                             # THE Odoo module, installed on each staging instance
│   └── hst_qa_probe/
│       ├── models/probe.py           # audit, fingerprint, run_scenario, run_committed, cleanup
│       ├── models/guards.py          # mail, http, commit, DDL patches, scoped to a transaction
│       ├── models/registry.py        # create-hook ledger for committed runs
│       └── data/neutralize.sql        # it neutralizes itself too
├── runner/                            # Python 3.12. One host, ours
│   ├── impact/                       # ast_python.py, xml_views.py, security_csv.py, manifest.py
│   ├── deploy/                       # per-host adapters: ssh.py, cloudpepper.py, odoo_sh.py
│   ├── planes/
│   │   ├── probe_client.py           # single-call scenario dispatch over JSON-RPC
│   │   └── ui.py                     # Playwright executor, evidence capture
│   ├── checkpoint/                   # pause, resume, re-entry gates, run-state persistence
│   ├── scenario/                     # schema.py, loader.py, expr.py, waits.py
│   ├── evidence/                     # shots.py, ledger.py, residue.py, report.py
│   └── jobs.py
├── flows/                             # THE shared asset
│   ├── sale/ stock/ purchase/ account/ mrp/ pos/ hr/ portal/
│   └── personas/                     # admin, app_user, minimal_user, portal
├── profiles/                          # 17.0.yml 18.0.yml 19.0.yml
├── clients/                           # per client: instance URL, db pattern, capabilities, stubs
└── docs/                             # scenario authoring, new-version drill, onboarding a client
```

Client-specific scenarios live in the *client's own repo* under `qa/scenarios/`, next to the code they test, versioned with it. Only the shared flow library and the profiles live here. Be strict about that boundary: a scenario in the wrong repo goes stale when the module changes.

| CONCERN | CHOICE | WHY THIS ONE |
|------------------------|--|---|
| Control plane | Express + TypeScript | The merge target's stack. No translation cost later |
| Runner | Python 3.12 | <code>ast</code> , Playwright, RPC. Use <code>python3.12</code> explicitly and build the venv with it, since <code>python3</code> on this machine is 3.10 |
| Probe | A plain Odoo module | The only thing that can hold a transaction open across a whole scenario. Version-branched in one file, driven by the profile |
| Transport to instances | JSON-RPC over HTTPS with an Odoo API key | The only channel that exists for every client, and on a managed host the only one there is. No feature may assume shell or Postgres access, because on Odoo.sh and Cloudpepper neither is on offer |
| Browser | Playwright for Python | Auto-waiting, trace viewer, reliable full-page screenshots. Odoo's own tour framework is the fallback for widgets Playwright cannot reach, not the primary |
| Queue | Postgres table with <code>SKIP LOCKED</code> , plus a per-client advisory lock | Run volume is tens per day. The lock is the important part, since instances cannot be shared |
| Report | Self-contained HTML + print CSS | Emailable, archivable, exports to PDF, no viewer to maintain |
| Bundle storage | S3 or R2, per-client prefix, 90-day lifecycle | Screenshots and traces get large fast. Keep <code>summary.json</code> , <code>residue.json</code> , and <code>preflight.json</code> indefinitely, expire the media |

THE AWKWARD PART, NAMED

Deploying a branch to a client staging instance has no universal mechanism. SSH plus `git pull` plus a module upgrade works where there is shell access. Cloudpepper has its own panel. Odoo.sh deploys from a branch push. So `runner/deploy/` is a per-host adapter directory, each one written once and reused across that host's clients.

This is the least elegant piece of the design and the one most likely to consume unplanned time. Budget for it explicitly rather than discovering it in week eight, and start with the adapter covering the most clients.

20 Roadmap and effort

Estimates assume one developer close to full time with you reviewing. Sequenced so each phase ships something usable alone, because a three-month project with nothing to show until the end does not survive contact with client work.

0 Static analysis, and the hygiene audit

1.5 weeks · needs no instance writes at all

- Python `ast` and XML parsers, the signal table as a lookup
- Three checks needing nothing else: missing `super()` , edited `noupdate` data, stored compute with no migration
- **The pre-flight audit as a standalone report**, run read-only across every client staging instance. UC-16

SHIPPED VALUE

Catches real bugs in week one, and tells you which client staging instances are currently capable of emailing real customers. The impact engine that later phases depend on is already built and exercised.

1 The probe module and the instance contract

2 weeks · the safety core, and the critical path

- `hst_qa_probe` : sentinel check, audit, fingerprint, savepoint execution with flush and invalidate
- The four guards, scoped to a transaction: mail, HTTP, commit, DDL
- Deliberate leak testing. Prove a scenario that tries to send mail is stopped, prove a rollback leaves no row, prove an unsafe instance is refused
- Install on one client's staging instance and verify against a database snapshot taken before and after

SHIPPED VALUE

The mechanism that makes everything else safe. Nothing above this phase can be trusted without it, so resist the temptation to build scenarios first.

2 Tiers 1 and 2, baseline, verdict, and pause

3.5 weeks · the core value

- Scenario schema with the required `tier` key, loader, expression evaluation
- Read-only probes: view resolution, report rendering, ACL matrix, query counts
- Nightly baseline, fingerprint capture and comparison, result diffing, deterministic verdict
- **Checkpointing, pause, and resume for tiers 1 and 2**, with the four re-entry gates and the "I need staging" request queue. Build it here rather than later: the rollback that ends every tier 2 scenario is already the checkpoint boundary, so it is nearly free now and a retrofit once tier 3 exists. Section 7
- First 25 flows in the library: sale and stock, widest reuse
- **The mechanical test plan**: selected steps with their provenance tags, posted to the task before execution. Needs no AI, and it is what makes the run reviewable from day one
- Per-client lock, deploy adapter for the most common host, Odoo chatter posting, advisory mode only

SHIPPED VALUE

Regression detection works, with zero residue on any client instance. This is the phase that delivers the actual promise, and it is worth a month in advisory mode before phase 3.

3 Tier 3, evidence, and cleanup

4 weeks · the part people see

- Playwright executor, logical selector resolution, the `waits.py` helpers
- Screenshot capture with element highlighting, the value ledger with readback
- **The creation registry, the reverse-order cleanup walker, and the residue report**, which is the extra work in-place testing costs over a throwaway database
- Tier 3 pause semantics: flow-boundary checkpoints, the abort-and-clean path for when somebody needs the instance now, and interim residue reporting for a run paused mid-suite
- Report generation, print stylesheet, bundle upload, retry and quarantine
- The weekly registry reconciliation sweep

SHIPPED VALUE

Ordered screenshots with input values, which is what makes this visible to project managers and clients. Also UC-14.

4 AI scenario proposal and coverage

2 weeks

- Propose scenarios from the diff plus the task, output valid YAML against the schema, including a defensible `tier`
- Ratification UI: accept, edit, reject, commit to the client repo as a PR
- Acceptance criteria parsing, `covers:` mapping, the `UNMET` verdict
- **Task interpretation:** restated requirement with line citations, the workflow summary, and the ambiguity list. Plus the assignment-time variant, which is the same component run before any code is written
- Approval modes, including the `auto` rule that holds a run only when the plan is uncertain
- The AI summary paragraph

SHIPPED VALUE

Suite growth stops being a chore. This is what gets 40 untested repos to meaningful coverage without a dedicated QA hire.

5 Dashboard merge, multi-version, blocking mode

2 weeks + ongoing

- Control plane merged into hst-pmo-dashboard, Odoo client and AI layer deduplicated
- Profiles for 17 and 19 brought up against the flow library on local throwaway instances
- Remaining deploy adapters, nightly runs per client, blocking enabled per client on the ten-green-tasks rule

SHIPPED VALUE

One place to see project status and QA status together.

REVISION 2'S PHASE 6 IS DELETED, NOT DEFERRED

It was one week of work for the clone path, and it unlocked UC-07's uninstall half, UC-08 as a true dry run, and UC-09. None of that is buildable against a managed host, so the week comes out of the roadmap and those three capabilities come out of the product. Section 18 records what a human now does instead.

| SCENARIO | CALENDAR | NOTES |
|--|-------------|--|
| One developer, close to full time | 15 weeks | Phases 0 to 5, sequential. There is no phase 6 to add |
| Two developers | 8.5 weeks | Split at phase 2 and 3, sharing only the scenario schema. Phase 1 cannot be parallelised, it is the critical path |
| Part time, one developer at 40 percent | ~7.5 months | Still worth it given the phase ordering, but the flow library will lag badly |
| Minimum useful subset | 7 weeks | Phases 0, 1, 2. Regression detection with zero residue, pause and resume working, no screenshots. Highest value per week spent |
| Two days, right now | — | UC-16 alone, the hygiene audit. Read-only, needs only RPC, and answers a question you cannot currently answer |

Against revision 2, the total moves by about a week. The clone path comes out, worth one week. Checkpointing, pause, and resume go in, worth about two. That is a fair trade even ignoring hosting reality, because the clone path served a minority of clients on a good day while pause serves every run on every client.

The larger change is not in the number. Revision 2 had a roadmap where four use cases waited on somebody negotiating database access from a hosting provider, which is the kind of dependency that quietly never resolves. Revision 3 has no such item: everything left in the plan can be built against an RPC endpoint and an API key, which every client already has.

Ongoing cost after the build is the number to plan around: a few hours a week on selectors and flows, two to four days per Odoo version bring-up, and a standing responsibility for keeping client staging instances audit-clean. Budget it as a line item rather than hoping it is zero, because a suite nobody maintains becomes a suite everybody skips.

21 Merging into the dashboard

Deliberately small, because the runner and the probe never move. Only the control plane folds in, and it was written in the dashboard's own stack for exactly this reason.

| WHAT MOVES | WHERE IT LANDS | WHAT IT REUSES INSTEAD OF DUPLICATING |
|------------------------------------|---|---|
| <code>api/src/routes/*</code> | <code>backend/src/routes/qa.routes.ts</code> | The existing <code>authMiddleware</code> and per-user Odoo session scope |
| Odoo access | Deleted | <code>services/odoo.service.ts</code> and <code>odoo-session.ts</code> already do this, including the API-key mode that 2FA accounts need |
| AI summary | <code>services/qa-summary.service.ts</code> | <code>ai-providers.ts</code> and its fallback chain, plus the three-layer key resolution. No new keys to manage |
| Queue, runs, residue, fingerprints | Their own tables | Nothing. Genuinely new state |
| Frontend | Four routes: run list, run detail, scenario registry, instance health | React Query, Recharts, the existing common components and Sidebar pattern |
| Runner and probe | Do not move | Runner keeps its <code>X-Runner-Token</code> auth, probe stays installed per instance |

THREE THINGS TO GET RIGHT AT MERGE TIME

- **Cache keys.** The dashboard's rule is that no Odoo-derived data is cached without being keyed by `uid`, because two users legitimately see different answers. QA run records are not per-user, they are per-task, so they must not go through those caches. Keep them in their own tables with their own access check, or they will either leak or be needlessly re-fetched.
- **Instance credentials are a new class of secret.** The gate holds an Odoo API key per client staging instance, which is not the same thing as the signed-in user's own credential. It belongs with the runner, not in the dashboard's per-user session store, and it should never be resolvable from a browser session. Worth deciding deliberately rather than by accident.
- **Branch.** Development on `hst-pmo-dashboard` happens on `staging` and Railway auto-deploys `main`. The QA merge lands on `staging` and reaches production when staging is merged, like everything else.

Worth noting what the dashboard gains beyond a new page. It already reads projects and tasks from Odoo. Adding QA outcomes means the executive view can answer questions it cannot today: which client's codebase is actually stable, how often work bounces back, whether the module a client wants extended has any coverage at all, and now also which client staging instances are healthy. That is a genuinely better dashboard, not just an extra tab.

22 Limits and risks

The concerns worth naming now, while the design can still absorb them. The top two are specific to testing in place and did not exist in revision 1.

| RISK | SEVERITY | HOW THE DESIGN HANDLES IT |
|--|----------|--|
| The gate is pointed at a production instance
A wrong URL in a client config file, and write-tests run against a live system | CRITICAL | Three independent controls, all of which must pass: the hand-placed sentinel naming the client, the database name allowlist, and the pre-flight audit. Opt-in by default means no mistake on our side makes production eligible. This is the risk that most justifies the instance contract being section 3 rather than an appendix |
| A staging instance is not neutralized
Live mail server, live crons, live integration credentials | CRITICAL | Audited before every run, never cached, and a failure refuses the run rather than warning. In-transaction guards on mail, HTTP, and commit as defence in depth. Expect the initial audit across your client base to fail on more instances than you would guess, which is itself the argument for building UC-16 first |
| Tier 3 leaves residue on client staging | HIGH | Bounded rather than eliminated, and that is the honest position. Existing records are never modified, the registry makes cleanup complete rather than best-effort, reversal handles what cannot be deleted, and everything remaining is named in the bundle. A QA company is available per client where even that is unacceptable |
| The temporal baseline is weaker than a twin, and there is no twin available at any price | CRITICAL | Raised from high in revision 2, because the escalation path that mitigated it is gone. Fingerprints make a stale comparison declare itself rather than producing a confident wrong answer, and the NEEDS RE-BASELINE verdict exists precisely for this. High-risk diffs take a fresh baseline immediately before deploying head, which collapses the drift window to minutes. Accepted, not solved. The bundle states the confidence it actually has |
| A paused run is never resumed
Someone takes the instance and forgets, leaving tier 3 residue uncleaned and results going stale | MED | New in revision 3, and the obvious failure mode of the pause feature. Every checkpoint carries <code>expires_at</code> ; past it the results are advisory and the run restarts. Paused runs are listed on the client's dashboard page sorted by age, the interim residue is already written so nothing is unaccounted for, and the weekly registry sweep reclaims anything cleanup never reached |
| Pause becomes the way people avoid the gate
Rather than waiting 20 minutes, everyone pauses everything | MED | Watch the ratio of paused time to run time per client as a first-class metric, not a log line. A client where runs spend more time paused than running has a capacity problem that pausing is hiding, and the answer there is a second staging instance from the host, not a cleverer gate |
| Flake erodes trust
Team starts ignoring or disabling the gate | CRITICAL | Advisory mode for weeks, per-client enablement on the ten-green-tasks rule, retry and quarantine, no wall-clock assertions, condition waits only. Treated as a design constraint, not a bug to fix later |
| Deploy mechanism differs per host | HIGH | Per-host adapters in <code>runner/deploy/</code> , written once each. Named openly as the least elegant part of the design and the most likely to consume unplanned time. Start with the adapter covering the most clients |
| Runs serialize per client
One staging instance cannot serve two commits | MED | Per-client lock, visible in the dashboard. A 12 to 20 minute run means roughly 20 runs a day per client, well above what five developers generate against one client. Accepted rather than solved |
| Existing code commits mid-flow and escapes rollback | MED | The commit guard raises, so the scenario fails loudly instead of leaking. The attempted commit is reported as a finding in its own right, because inline commits in business logic are a defect worth fixing. Expect to find a few in older custom modules |
| A long transaction blocks a person using | MED | One scenario per transaction so they last seconds, plus <code>lock_timeout</code> and <code>statement_timeout</code> on the probe's work. A timeout reports INCONCLUSIVE rather than failing the task |

| RISK | SEVERITY | HOW THE DESIGN HANDLES IT |
|--|----------|---|
| staging | | |
| "All possible scenarios" is not achievable | REFRAME | Replaced with risk-based selection plus an accumulating suite. Coverage of what a change can plausibly break, measured and reported honestly, rather than a claim of completeness |
| POS and heavy-JS screens automate poorly | HIGH | Accepted. Those flows get evidence capture and a console error budget rather than deep assertions, and the report says plainly where automated confidence is low so a human knows where to look. They also carry the worst residue profile, which is a second reason to keep them few |

What it will not do, stated plainly

- **It cannot make tier 3 residue-free.** A browser click commits, and Odoo will not let you delete a posted entry. The design bounds and reports residue rather than pretending to eliminate it, and anyone who needs zero residue on staging should read section 6 and choose the QA company option knowing what it costs.
- **It cannot test a module uninstall, verify a migration script by running it, or dry-run a version upgrade.** All three needed a database that could be thrown away afterwards, and a managed host does not provide one. The gate reads those changes statically and reports what it can see, but a human with a backup does the actual work. This is the largest capability difference between revision 2 on paper and revision 3 in reality, and nobody should discover it mid-project.
- **It cannot judge whether a feature is a good idea,** whether the UX is right, or whether an ambiguous acceptance criterion was interpreted the way the client meant. Human review does not go away. It gets cheaper and better targeted.
- **It cannot find a bug in a path nobody has described.** Coverage is exactly the union of the flow library and the client's ratified scenarios. The honest framing is that it makes every bug found *once* a bug that can never reach a client *twice*.
- **It will not replace testing on the client's production instance** for changes involving hardware, real payment terminals, real printers, or real device integrations.
- **It does not make the first month faster.** The gate adds 15 to 20 minutes to a task's cycle and, early on, will occasionally be wrong. The return arrives when the suite has depth, which is around task thirty for a given client.

23 Is it working?

Four numbers, tracked from the first advisory run so there is a real before and after rather than a feeling. Everything else is vanity.

THE ONE THAT MATTERS

Client-reported regressions per month

Bugs in *existing* functionality, reported by a client, per client per month. This is the number the gate exists to reduce. Start counting it now, before the build, or you will have nothing to compare against.

EFFICIENCY

Bounce rate from PM Review

Share of tasks a project manager sends back. Should fall, because the obvious failures are caught earlier. If it does not move, the gate is testing the wrong things.

TRUST

False-positive rate

Share of blocking verdicts a developer overturns as wrong. Must stay under about 5 percent. Above that, turn blocking off for that client and fix the suite. This is the number that decides whether the tool is used or resented.

SAFETY

Residue records per run, and audit pass rate

New in this revision, and the licence to keep running against client instances. Residue should trend toward zero as cleanup rules improve. Audit pass rate across instances should reach 100 percent and stay there.

One operational number sits alongside these rather than among them, because it measures the instance rather than the gate: **the ratio of paused time to running time per client**. It is how you find out that a client's single staging instance has become the bottleneck, which is a capacity conversation with the host rather than anything the gate can fix.

One softer signal is worth watching too: whether developers start running the gate voluntarily before they are asked to. That happens when it is faster than clicking through staging themselves, and it is the point at which the tool has actually landed.